

Time Series Analysis

Md. Mostakim

Student of Applied Statistics and Data Science
Institute of Applied Statistics and Data Science
University of Dhaka



Section 1

Time Series Analysis

From raw data to useful forecasts

1. Explore the time index and frequency
2. Visualize patterns, trends, and seasonality
3. Diagnose stationarity and dependence
4. Transform non-stationary series
5. Build WN, RW, AR, and MA models
6. Compare models and forecast

Core workflow: *Observe* $\rightarrow$ *Diagnose* $\rightarrow$ *Transform* $\rightarrow$ *Model*
 $\rightarrow$ *Validate* $\rightarrow$ *Forecast*

By the end of this lecture, you should be able to:

- ▶ Construct and inspect `ts` objects in R
- ▶ Recognize trend, seasonality, persistence, and stationarity
- ▶ Interpret ACF plots and lagged dependence
- ▶ Explain WN, RW, AR, and MA models
- ▶ Estimate models with `arima()`
- ▶ Produce forecasts and compare models using AIC/BIC

The four models at a glance

Model	Key idea	Stationary?	ACF signature
WN	No serial dependence		Near zero after lag 0
RW	Current value = previous value + noise		Very slow decay
AR(1)	Depends on previous value	if $ \phi < 1$	Gradual/geometric decay
MA(1)	Depends on current & previous shocks		Cuts off after lag 1

The four models at a glance (cont.)

Exam cue: *ACF shape is often the fastest clue to model structure.*

Section 2

Exploring raw time series

Exploring raw time series

The most common first step when conducting time series analysis is to display your time series dataset in a visually intuitive format. The most useful way to view raw time series data in R is to use the `print()` command, which displays the Start, End, and Frequency of your data along with the observations.

Another useful command for viewing time series data in R is the `length()` function, which tells you the total number of observations in your data.

Some datasets are very long, and previewing a subset of data is more suitable than displaying the entire series. The `head(____, n = ____)` and `tail(____, n = ____)` functions, in which `n` is the number of items to display, focus on the first and last few elements of a given dataset respectively.

Exploring raw time series (cont.)

In this exercise, you'll explore the famous River Nile annual streamflow data, Nile. This time series dataset includes some metadata information. When calling `print(Nile)`, note that `Start = 1871` indicates that 1871 is the year of the first annual observation, and `End = 1970` indicates 1970 is the year of the last annual observation

```
# List the number of observations in the Nile dataset  
length(Nile)
```

```
## [1] 100
```

```
# Display the first 10 elements of the Nile dataset  
head(Nile)
```

Exploring raw time series (cont.)

```
## Time Series:  
## Start = 1871  
## End = 1876  
## Frequency = 1  
## [1] 1120 1160 963 1210 1160 1160
```

```
# Display the last 12 elements of the Nile dataset  
tail(Nile)
```

```
## Time Series:  
## Start = 1965  
## End = 1970  
## Frequency = 1  
## [1] 912 746 919 718 714 740
```

Section 3

Basic time series plots

Basic time series plots

While simple commands such as `print()`, `length()`, `head()`, and `tail()` provide crucial information about your time series data, another very useful way to explore any data is to generate a plot.

In this exercise, you will plot the River Nile annual streamflow data using the `plot()` function. For time series data objects such as `Nile`, a Time index for the horizontal axis is typically included. From the previous exercise, you know that this data spans from 1871 to 1970, and horizontal tick marks are labeled as such. The default label of "Time" is not very informative. Since these data are annual measurements, you should use the label "Year". While you're at it, you should change the vertical axis label to "River Volume (10^9 m^3)".

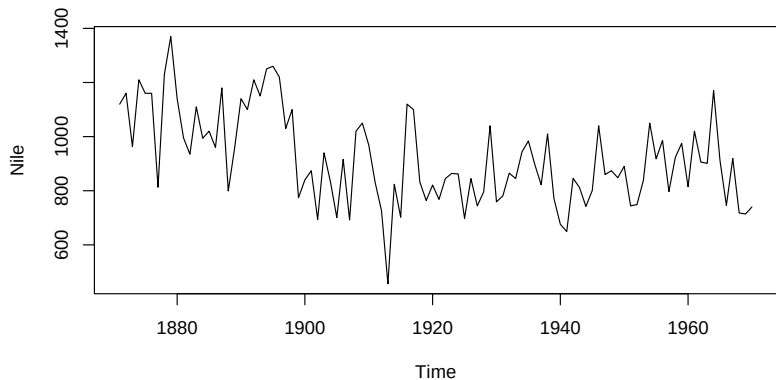
Additionally, it helps to have an informative title, which can be set using the argument `main`. For your purposes, a useful title for this figure would be "Annual River Nile Volume at Aswan, 1871-1970".

Basic time series plots (cont.)

Finally, the default plotting type for time series objects is "l" for line. Connecting consecutive observations can help make a time series plot more interpretable. Sometimes it is also useful to include both the observations points as well as the lines, and we instead use "b" for both.

```
# Plot the Nile data  
plot(Nile)
```

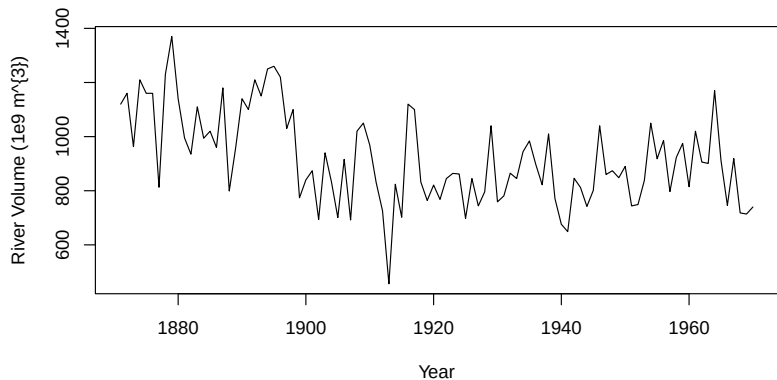
Basic time series plots (cont.)



Basic time series plots (cont.)

```
# Plot the Nile data with xlab and ylab arguments  
plot(Nile, xlab = "Year",  
      ylab = "River Volume (1e9 m3)")
```

Basic time series plots (cont.)

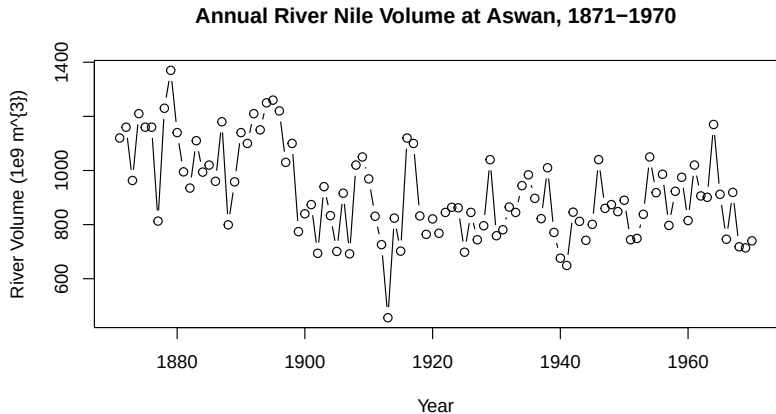


Basic time series plots (cont.)

Plot the Nile data with xlab, ylab, main, and type arguments

```
plot(Nile, xlab = "Year",  
      ylab = "River Volume (1e9 m{3})",  
      main = "Annual River Nile Volume at Aswan, 1871-1970",  
      type = "b")
```

Basic time series plots (cont.)



Section 4

What does the time index tell us?

What does the time index tell us?

Some data are naturally evenly spaced by time. The time series `discrete_data` shown in the top figure has 20 observations, with one observation appearing at each of the discrete time indices 1 through 20. Discrete time indexing is appropriate for `discrete_data`.

The time series `continuous_series` shown in the bottom figure also has 20 observations, it is following the same periodic pattern as `discrete_data`, but its observations are not evenly spaced. Its first, second, and last observations were observed at times 1.210322, 1.746137, and 20.180524, respectively. Continuous time indexing is natural for `continuous_series`, however, the observations are approximately evenly spaced, with about 1 observation observed per time unit. Let's investigate using a discrete time indexing for `continuous_series`.

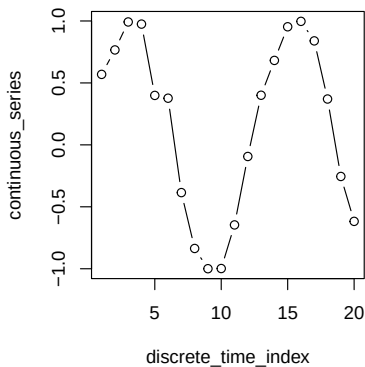
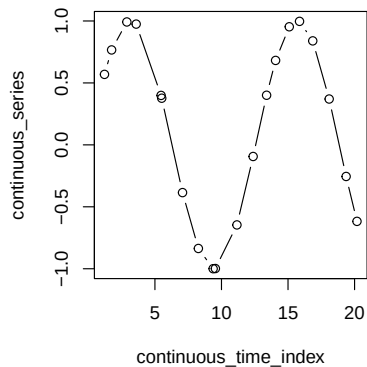
What does the time index tell us? (cont.)

```
# Plot the continuous_series using continuous time indexing
par(mfrow=c(1,2))
plot(continuous_time_index,continuous_series,
     type = "b")

# Make a discrete time index using 1:20
discrete_time_index <- 1:20

# Now plot the continuous_series using discrete time indexing
plot(discrete_time_index,continuous_series,
     type = "b")
```

What does the time index tell us? (cont.)



Section 5

Sampling & Time Structure

Identifying the sampling frequency

In addition to viewing your data and plotting over time, there are several additional operations that can be performed on time series datasets.

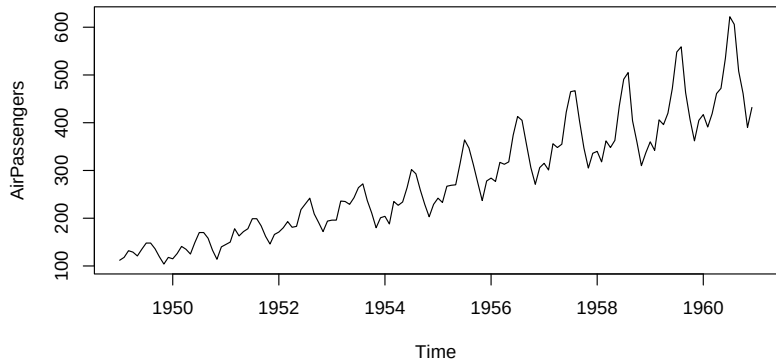
The `start()` and `end()` functions return the time index of the first and last observations, respectively. The `time()` function calculates a vector of time indices, with one element for each time index on which the series was observed.

The `deltat()` function returns the fixed time interval between observations and the `frequency()` function returns the number of observations per unit time. Finally, the `cycle()` function returns the position in the cycle of each observation.

In this exercise, you'll practice applying these functions to the `AirPassengers` dataset, which reports the monthly total international airline passengers (in thousands) from 1949 to 1960.

Identifying the sampling frequency (cont.)

```
# Plot AirPassengers  
plot(AirPassengers)
```



Identifying the sampling frequency (cont.)

```
# View the start and end dates of AirPassengers
```

```
start(AirPassengers)
```

```
## [1] 1949    1
```

```
end(AirPassengers)
```

```
## [1] 1960   12
```

```
# Use time(), deltat(), frequency(), and cycle() with AirPas
```

```
time(AirPassengers)
```

Identifying the sampling frequency (cont.)

##		Jan	Feb	Mar	Apr	May	Ju
##	1949	1949.000	1949.083	1949.167	1949.250	1949.333	1949.41
##	1950	1950.000	1950.083	1950.167	1950.250	1950.333	1950.41
##	1951	1951.000	1951.083	1951.167	1951.250	1951.333	1951.41
##	1952	1952.000	1952.083	1952.167	1952.250	1952.333	1952.41
##	1953	1953.000	1953.083	1953.167	1953.250	1953.333	1953.41
##	1954	1954.000	1954.083	1954.167	1954.250	1954.333	1954.41
##	1955	1955.000	1955.083	1955.167	1955.250	1955.333	1955.41
##	1956	1956.000	1956.083	1956.167	1956.250	1956.333	1956.41
##	1957	1957.000	1957.083	1957.167	1957.250	1957.333	1957.41
##	1958	1958.000	1958.083	1958.167	1958.250	1958.333	1958.41
##	1959	1959.000	1959.083	1959.167	1959.250	1959.333	1959.41
##	1960	1960.000	1960.083	1960.167	1960.250	1960.333	1960.41
##		Sep	Oct	Nov	Dec		
##	1949	1949.667	1949.750	1949.833	1949.917		
##	1950	1950.667	1950.750	1950.833	1950.917		

Identifying the sampling frequency (cont.)

```
## 1951 1951.667 1951.750 1951.833 1951.917
## 1952 1952.667 1952.750 1952.833 1952.917
## 1953 1953.667 1953.750 1953.833 1953.917
## 1954 1954.667 1954.750 1954.833 1954.917
## 1955 1955.667 1955.750 1955.833 1955.917
## 1956 1956.667 1956.750 1956.833 1956.917
## 1957 1957.667 1957.750 1957.833 1957.917
## 1958 1958.667 1958.750 1958.833 1958.917
## 1959 1959.667 1959.750 1959.833 1959.917
## 1960 1960.667 1960.750 1960.833 1960.917
```

```
deltat(AirPassengers)
```

```
## [1] 0.08333333
```

```
frequency(AirPassengers)
```

Identifying the sampling frequency (cont.)

```
## [1] 12
```

```
cycle(AirPassengers)
```

##		Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
##	1949	1	2	3	4	5	6	7	8	9	10	11	12
##	1950	1	2	3	4	5	6	7	8	9	10	11	12
##	1951	1	2	3	4	5	6	7	8	9	10	11	12
##	1952	1	2	3	4	5	6	7	8	9	10	11	12
##	1953	1	2	3	4	5	6	7	8	9	10	11	12
##	1954	1	2	3	4	5	6	7	8	9	10	11	12
##	1955	1	2	3	4	5	6	7	8	9	10	11	12
##	1956	1	2	3	4	5	6	7	8	9	10	11	12
##	1957	1	2	3	4	5	6	7	8	9	10	11	12
##	1958	1	2	3	4	5	6	7	8	9	10	11	12
##	1959	1	2	3	4	5	6	7	8	9	10	11	12
##	1960	1	2	3	4	5	6	7	8	9	10	11	12

Section 6

Building ts Objects

Creating a time series object with `ts()`

The function `ts()` can be applied to create time series objects. A time series object is a vector (univariate) or matrix (multivariate) with additional attributes, including time indices for each observation, the sampling frequency and time increment between observations, and the cycle length for periodic data. Such objects are of the `ts` class, and represent data that has been observed at (approximately) equally spaced time points. Now you will create time series objects yourself.

The advantage of creating and working with time series objects of the `ts` class is that many methods are available for utilizing time series attributes, such as time index information. For example, as you've seen in earlier exercises, calling `plot()` on a `ts` object will automatically generate a plot over time.

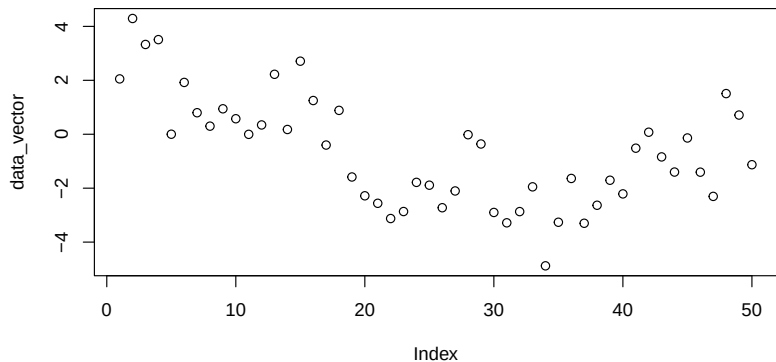
Creating a time series object with `ts()` (cont.)

In this exercise, you'll familiarize yourself with the `ts` class by encoding some time series data (saved as `data_vector`) into `ts` and exploring the result. Your time series `data_vector` starts in the year 2004 and has 4 observations per year (i.e. it is quarterly data).

```
# Use plot() to view data_vector
```

```
plot(data_vector)
```

Creating a time series object with ts() (cont.)



Creating a time series object with ts() (cont.)

```
# Convert data_vector to a ts object with start = 2004 and frequency = 4
time_series <- ts(data_vector, start = c(2004, 1), frequency = 4)

# Use print() and plot() to view time_series
print(time_series)
```

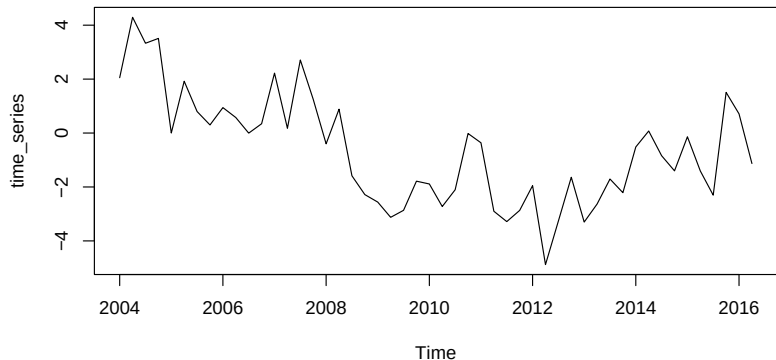
##		Qtr1	Qtr2	Qtr3	Qtr4
##	2004	2.0521941073	4.2928852797	3.3294132944	3.5085950121
##	2005	0.0009576938	1.9217186345	0.7978134128	0.2999543201
##	2006	0.9435687536	0.5748283388	-0.0034005903	0.3448649121
##	2007	2.2229761136	0.1763144576	2.7097622770	1.2501948121
##	2008	-0.4007164754	0.8852732121	-1.5852420266	-2.2829278121
##	2009	-2.5609531290	-3.1259963754	-2.8660295895	-1.7847009121
##	2010	-1.8894912908	-2.7255351194	-2.1033141800	-0.0174256121
##	2011	-0.3613204151	-2.9008403327	-3.2847440927	-2.8684594121
##	2012	-1.9505074437	-4.8801892525	-3.2634605353	-1.6396062121

Creating a time series object with ts() (cont.)

```
## 2013 -3.3012575840 -2.6331245433 -1.7058354022 -2.2119825
## 2014 -0.5170595186  0.0752508095 -0.8406994716 -1.4022683
## 2015 -0.1382114230 -1.4065954703 -2.3046941055  1.5073891
## 2016  0.7118679477 -1.1300519022
```

```
plot(time_series)
```

Creating a time series object with ts() (cont.)



Breakdown of the *start = c(1949, 1)* Syntax

The `c(1949, 1)` part is a vector with two specific arguments:

Creating a time series object with `ts()` (cont.)

- ▶ The First Number (1949): Represents the major time unit, which is the calendar year.
- ▶ The Second Number (1): Represents the minor time unit, which is the cycle/period within that year.

Examples Based on Frequency

The meaning of that second number changes depending on the frequency you set for your data:

- ▶ **Monthly** (frequency = 12): `c(1949, 1)` means January 1949. If you wrote `c(1949, 6)`, it would mean June 1949.
- ▶ **Quarterly** (frequency = 4): `c(1949, 1)` means Quarter 1 (Jan-Mar) of 1949. If you wrote `c(1949, 3)`, it would mean Quarter 3.
- ▶ **Weekly** (frequency = 52): `c(1949, 1)` means Week 1 of 1949.

Testing whether an object is a time series

When you work to create your own datasets, you can build them as `ts` objects. Recall the dataset `data_vector` from the previous exercise, which was just a vector of numbers, and `time_series`, the `ts` object you created from `data_vector` using the `ts()` function and information regarding the start time and the observation frequency. As a reminder, `data_vector` and `time_series` are shown in the plot on the right.

When you use datasets from others, such as those included in an R package, you can check whether they are `ts` objects using the `is.ts()` command. The result of the test is either `TRUE` when the data is of the `ts` class, or `FALSE` if it is not.

In this exercise, you'll explore the class of the datasets you've been using throughout this chapter.

```
# Check whether data_vector and time_series are ts objects  
is.ts(data_vector)
```

Testing whether an object is a time series (cont.)

```
## [1] FALSE
```

```
is.ts(time_series)
```

```
## [1] TRUE
```

```
# Check whether Nile is a ts object
```

```
is.ts(Nile)
```

```
## [1] TRUE
```

```
# Check whether AirPassengers is a ts object
```

```
is.ts(AirPassengers)
```

```
## [1] TRUE
```

Plotting a time series object

It is often very useful to plot data we are analyzing, as is the case when conducting time series analysis. If the dataset under study is of the `ts` class, then the `plot()` function has methods that automatically incorporate time index information into a figure.

Let's consider the `eu_stocks` dataset (available in R by default as `EuStockMarkets`). This dataset contains daily closing prices of major European stock indices from 1991-1998, specifically, from Germany (DAX), Switzerland (SMI), France (CAC), and the UK (FTSE). The data were observed when the markets were open, so there are no observations on weekends and holidays. We will proceed with the approximation that this dataset has evenly spaced observations and is a four dimensional time series.

To conclude this chapter, this exercise asks you to apply several of the functions you've already learned to this new dataset.

Plotting a time series object (cont.)

```
eu_stocks <- EuStockMarkets
```

```
# Check whether eu_stocks is a ts object  
is.ts(eu_stocks)
```

```
## [1] TRUE
```

```
# View the start, end, and frequency of eu_stocks  
start(eu_stocks)
```

```
## [1] 1991 130
```

```
end(eu_stocks)
```

```
## [1] 1998 169
```

```
frequency(eu_stocks)
```

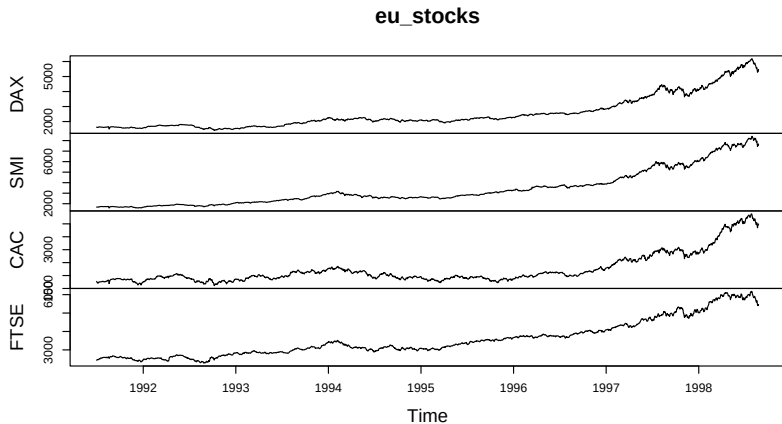
Plotting a time series object (cont.)

```
## [1] 260
```

```
# Generate a simple plot of eu_stocks
```

```
plot(eu_stocks)
```

Plotting a time series object (cont.)



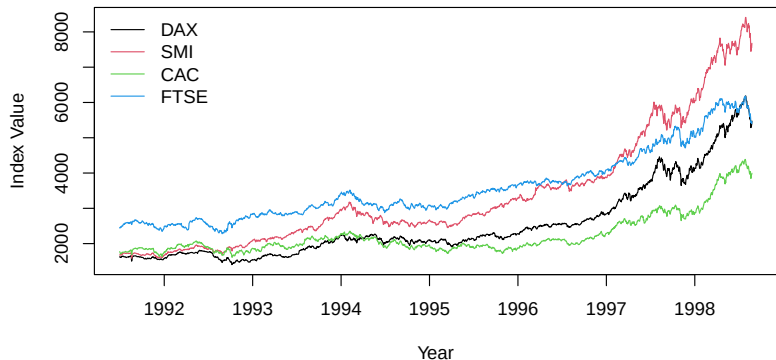
Plotting a time series object (cont.)

```
# Use ts.plot with eu_stocks
ts.plot(eu_stocks, col = 1:4,
        xlab = "Year",
        ylab = "Index Value",
        main = "Major European Stock Indices, 1991-1998")

# Add a legend to your ts.plot
legend("topleft", colnames(eu_stocks),
       lty = 1, col = 1:4, bty = "n")
```

Plotting a time series object (cont.)

Major European Stock Indices, 1991–1998



Section 7

Missing Values & Data Preparation

2. Convert to a formal time series object

```
custom_AirPassengers <- ts(air_data,  
                           start = c(1949, 1),  
                           frequency = 12)
```

3. View the result

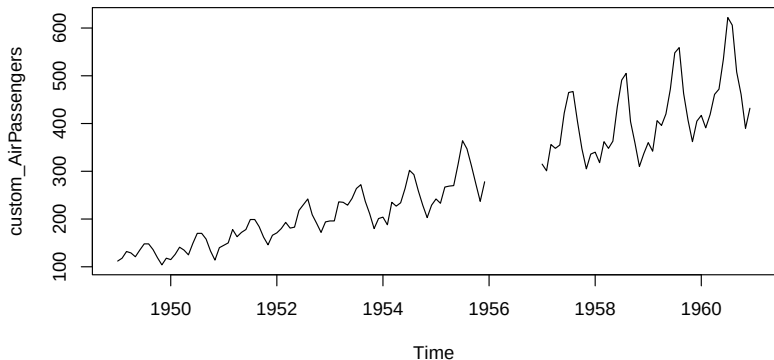
```
head(custom_AirPassengers)
```

```
##           Jan Feb Mar Apr May Jun  
## 1949 112 118 132 129 121 135
```

Plot the custom_AirPassengers data

```
plot(custom_AirPassengers)
```

Missing Values (cont.)



```
# Compute the mean of AirPassengers  
mean(custom_AirPassengers, na.rm=T)
```

Missing Values (cont.)

```
## [1] 275.9394
```

```
# Impute mean values to NA in AirPassengers
```

```
custom_AirPassengers[is.na(custom_AirPassengers)] <- mean(custom_AirPassengers)
```

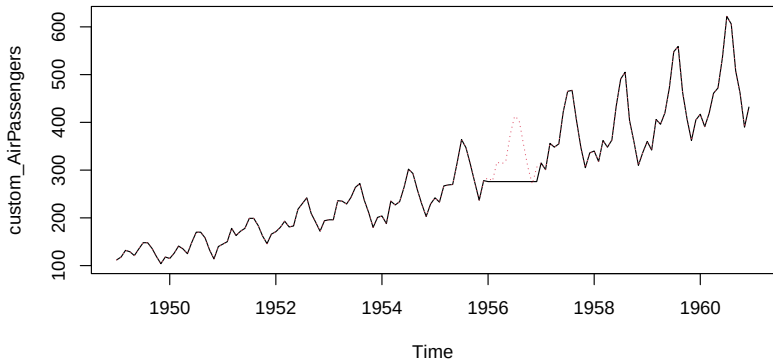
```
# Generate another plot of AirPassengers
```

```
plot(custom_AirPassengers)
```

```
# Add the complete AirPassengers data to your plot
```

```
points(AirPassengers, type = "l", col = 2, lty = 3)
```

Missing Values (cont.)



Section 8

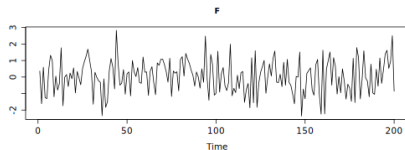
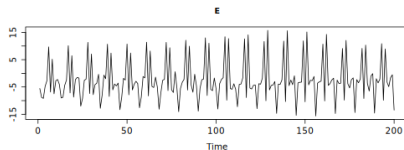
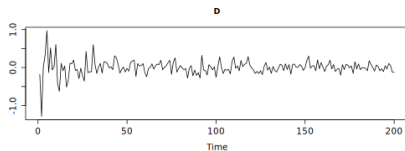
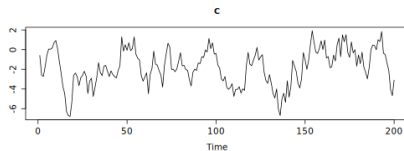
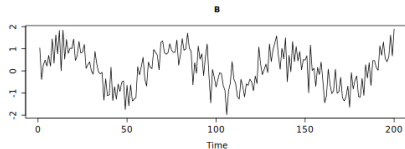
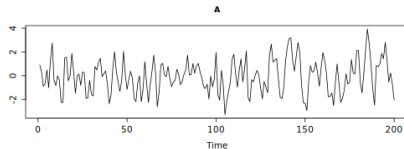
Trend, Transformation & Differencing

Random or not random?

Some time series exhibit predictability when strong periodic or seasonal patterns are present. Other time series exhibit predictability when positive autocorrelation - or correlation among neighboring observations - induces what appear as short-term trends.

Examine the time series plots below. Which pair of time series do NOT appear predictable?

Random or not random? (cont.)

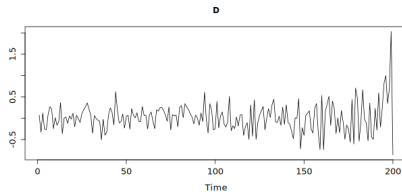
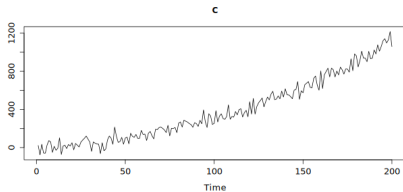
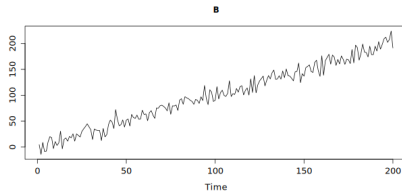
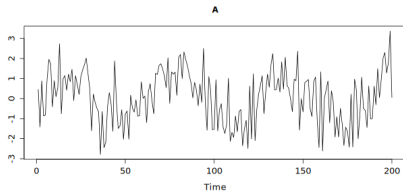


► **Answer:** ✓ D and F

Common types of trends in time series data include persistent growth or decay over time, increasing variability over time, and periodic or seasonal patterns, that is regularly spaced peaks and troughs appearing throughout the data.

Which time series exhibits an approximately linear growth pattern over time?

Name that trend (cont.)

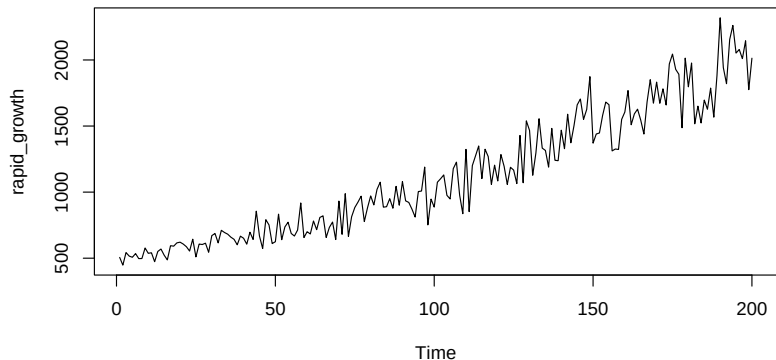


► **Answer:** ✓ B

Removing trends in variability via the logarithmic transformation

The logarithmic function $\log()$ is a data transformation that can be applied to positively valued time series data. It slightly shrinks observations that are greater than one towards zero, while greatly shrinking very large observations. This property can stabilize variability when a series exhibits increasing variability over time. It may also be used to linearize a rapid growth pattern over time.

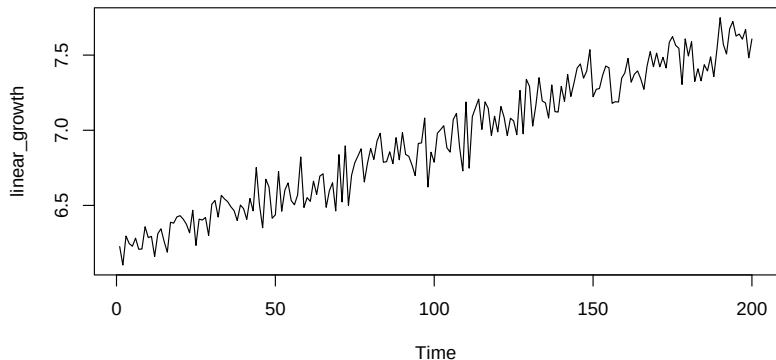
Removing trends in variability via the logarithmic transformation (cont.)



Removing trends in variability via the logarithmic transformation (cont.)

```
# Log rapid_growth  
linear_growth <- log(rapid_growth)  
  
# Plot linear_growth using ts.plot()  
  
ts.plot(linear_growth)
```

Removing trends in variability via the logarithmic transformation (cont.)

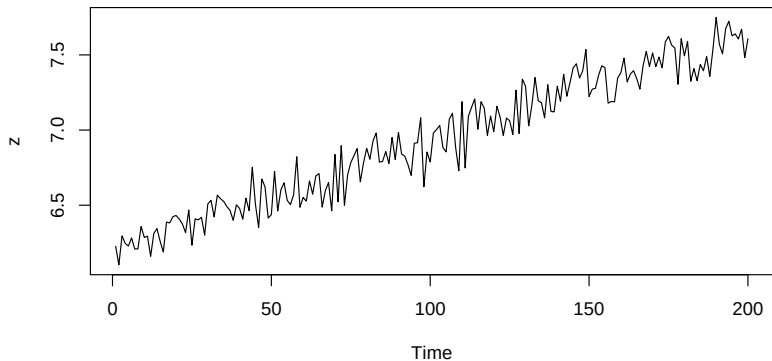


Removing trends in level by differencing

The first difference transformation of a time series consists of the differences (changes) between successive observations over time, that is $z[t] - z[t-1]$.

Differencing a time series can remove a time trend. The function `diff()` will calculate the first difference or change series. A difference series lets you examine the increments or changes in a given time series. It always has one fewer observations than the original series.

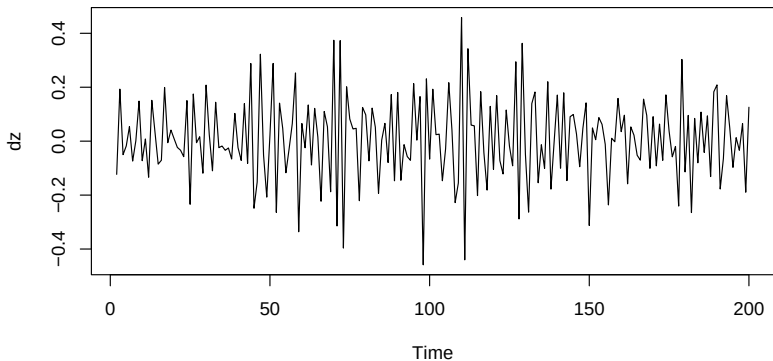
Removing trends in level by differencing (cont.)



Removing trends in level by differencing (cont.)

```
# Generate the first difference of z  
dz <- diff(z)  
  
# Plot dz  
ts.plot(dz)
```

Removing trends in level by differencing (cont.)



Removing trends in level by differencing (cont.)

```
# View the length of z and dz, respectively
```

```
length(z)
```

```
## [1] 200
```

```
length(dz)
```

```
## [1] 199
```

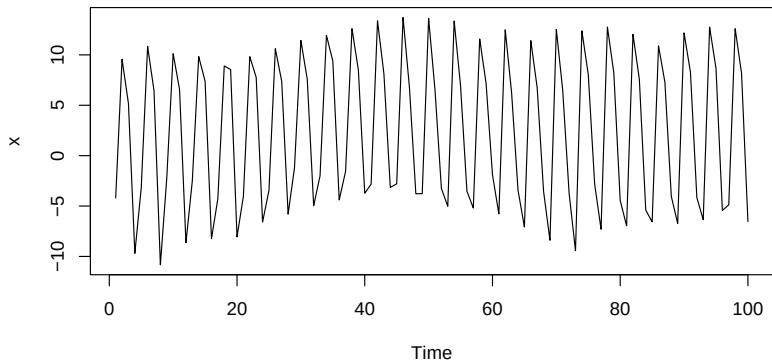
By removing the long-term time trend, you can view the amount of change from one observation to the next.

Removing seasonal trends with seasonal differencing

For time series exhibiting seasonal trends, seasonal differencing can be applied to remove these periodic patterns. For example, monthly data may exhibit a strong twelve month pattern. In such situations, changes in behavior from year to year may be of more interest than changes from month to month, which may largely follow the overall seasonal pattern.

The function `diff(..., lag = s)` will calculate the lag s difference or length s seasonal change series. For monthly or quarterly data, an appropriate value of s would be 12 or 4, respectively. The `diff()` function has `lag = 1` as its default for first differencing. Similar to before, a seasonally differenced series will have s fewer observations than the original series.

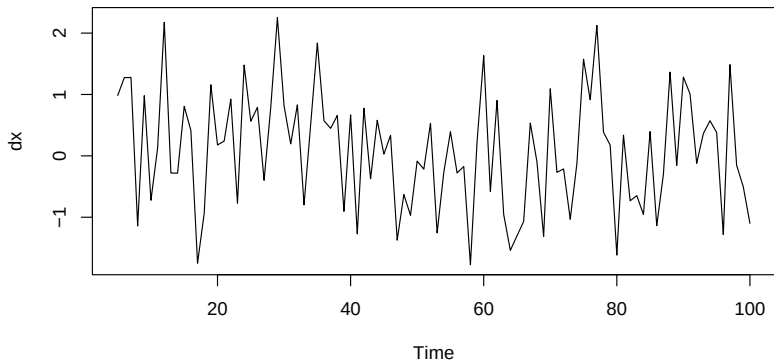
Removing seasonal trends with seasonal differencing (cont.)



Removing seasonal trends with seasonal differencing (cont.)

```
# Generate a diff of x with lag = 4. Save this to dx  
dx <- diff(x, lag = 4)  
  
# Plot dx  
ts.plot(dx)
```

Removing seasonal trends with seasonal differencing (cont.)



Removing seasonal trends with seasonal differencing (cont.)

View the length of x and dx, respectively

```
length(x)
```

```
## [1] 100
```

```
length(dx)
```

```
## [1] 96
```

Once again differencing allows you to remove the longer-term time trend - in this case, seasonal volatility - and focus on the change from one period to another.

Section 9

White Noise

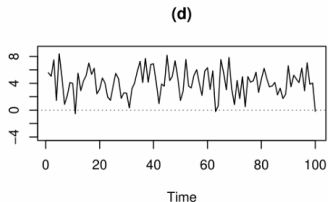
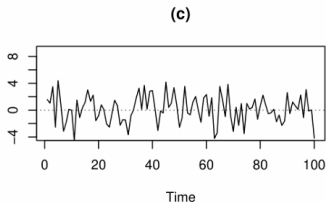
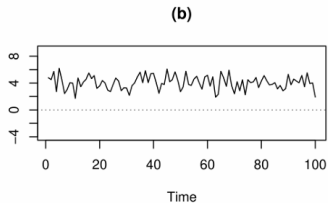
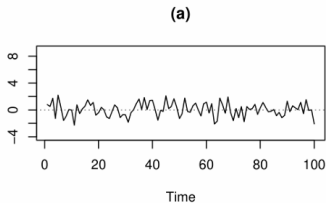
The White noise (WN) model

White Noise (WN) is the simplest example of a stationary process.

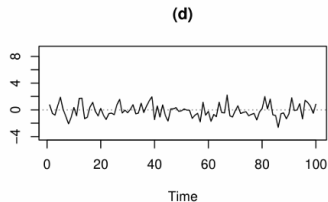
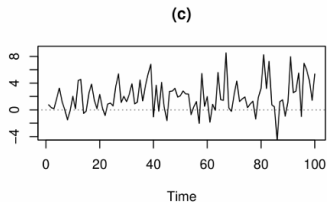
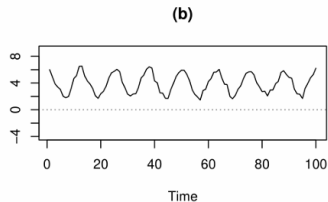
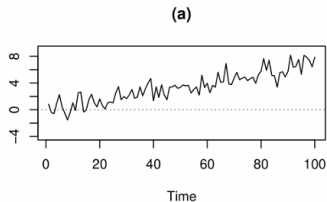
A weak white noise process has:

- ▶ A fixed, constant mean.
- ▶ A fixed, constant variance.
- ▶ No correlation over time

Time series plots of White Noise



Time series plots of White Noise?



Answer: ✓ d.

Simulate the white noise model

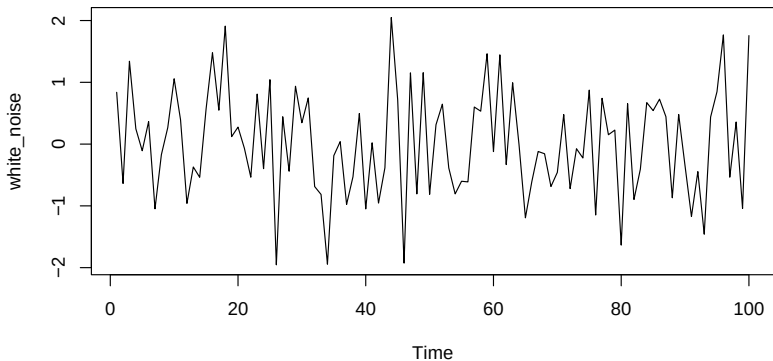
The white noise (WN) model is a basic time series model. It is also a basis for the more elaborate models we will consider. We will focus on the simplest form of WN, independent and identically distributed data. The `arima.sim()` function can be used to simulate data from a variety of time series models. ARIMA is an abbreviation for the **autoregressive integrated moving average** class of models we will consider throughout this course.

An **ARIMA(p, d, q)** model has three parts, the autoregressive order p , the order of integration (or differencing) d , and the moving average order q . We will detail each of these parts soon, but for now we note that the **ARIMA(0, 0, 0)** model, i.e., with all of these components zero, is simply the WN model.

Simulate the white noise model (cont.)

```
# Simulate a WN model with list(order = c(0, 0, 0))  
white_noise <- arima.sim(model = list(order = c(0, 0, 0)),  
                           n = 100)  
  
# Plot your white_noise data  
ts.plot(white_noise)
```

Simulate the white noise model (cont.)

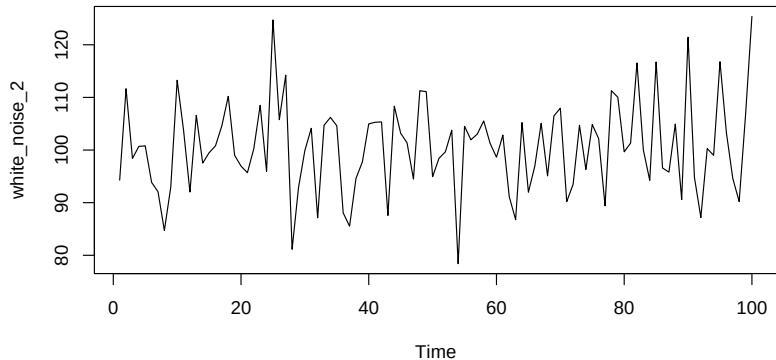


Simulate the white noise model (cont.)

```
# Simulate from the WN model with: mean = 100, sd = 10
white_noise_2 <- arima.sim(model = list(order = c(0, 0, 0)),
                           n = 100,
                           mean = 100,
                           sd = 10)

# Plot your white_noise_2 data
ts.plot(white_noise_2)
```

Simulate the white noise model (cont.)

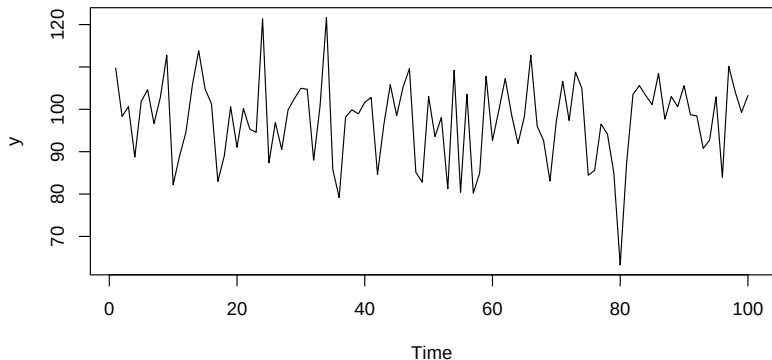


Estimate the white noise model

For a given time series y we can fit the white noise (WN) model using the `arma(..., order = c(0, 0, 0))` function. Recall that the WN model is an **ARIMA(0,0,0)** model. Applying the `arma()` function returns information or output about the estimated model. For the WN model this includes the estimated mean, labeled `intercept`, and the estimated variance, labeled `sigma^2`.

In this exercise, you'll explore the qualities of the WN model. What is the estimated mean? Compare this with the sample mean using the `mean()` function. What is the estimated variance? Compare this with the sample variance using the `var()` function.

Estimate the white noise model (cont.)



```
# Fit the WN model to y using the arima command  
arima(y, order = c(0, 0, 0))
```

Estimate the white noise model (cont.)

```
##  
## Call:  
## arima(x = y, order = c(0, 0, 0))  
##  
## Coefficients:  
##      intercept  
##      97.5428  
## s.e.      0.9697  
##  
## sigma^2 estimated as 94.03:  log likelihood = -369.08, a  
# Calculate the sample mean and sample variance of y  
mean(y)  
  
## [1] 97.54284
```

Estimate the white noise model (cont.)

```
var(y)
```

```
## [1] 94.98174
```

Section 10

The random walk (RW) model

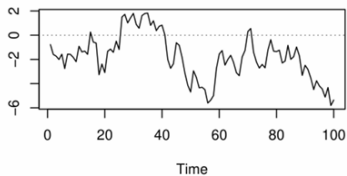
Random Walk (RW) is a simple example of a non-stationary process.

A random walk has:

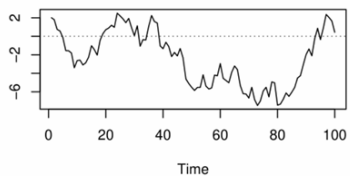
- ▶ No specified mean or variance.
- ▶ Strong dependence over time.
- ▶ Its changes or increments are white noise (WN).

Time series plots of Random Walk

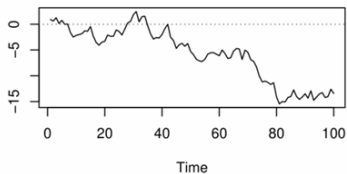
(a)



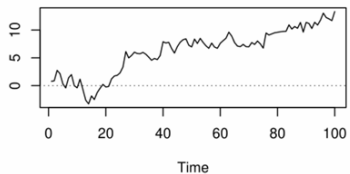
(b)



(c)



(d)



$$Today = Yesterday + Noise$$

More formally:

$$Y_t = Y_{t-1} + \epsilon_t$$

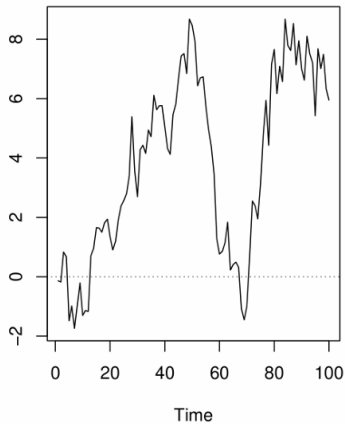
where ϵ_t is mean zero white noise (WN).

- ▶ Simulation requires an initial point Y_0 .
- ▶ Only one parameter, the WN variance σ_ϵ^2 .

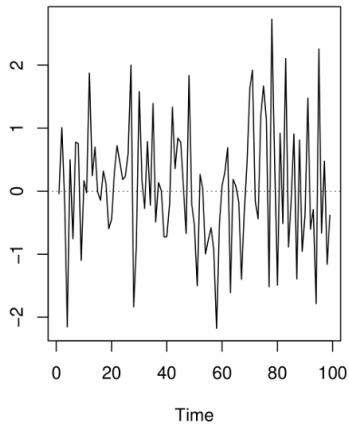
As $Y_t - Y_{t-1} = \epsilon_t \rightarrow \text{diff}(Y)$ is WN

Random walk

Y



diff(Y)



The random walk with a drift:

$$Today = Constant + Yesterday + Noise$$

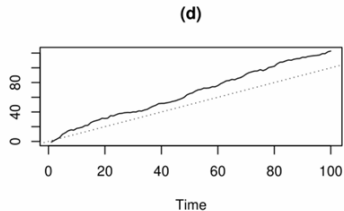
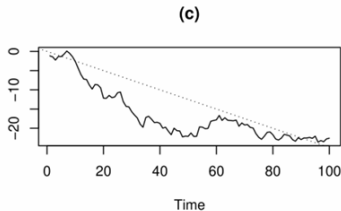
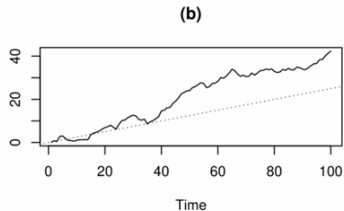
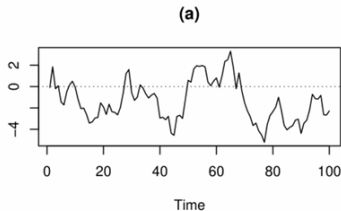
More formally:

$$Y_t = c + Y_{t-1} + \epsilon_t$$

where ϵ_t is mean zero white noise (WN).

- ▶ Two parameters, the constant c , and the WN variance σ_ϵ^2 .
- ▶ $Y_t - Y_{t-1} = ? \rightarrow$ WN with mean c !

Time series plots of Random Walk with drift



Main difference between random walk model with and without drift

Feature	Random Walk Without Drift	Random Walk With Drift
Model	$Y_t = Y_{t-1} + \varepsilon_t$	$Y_t = c + Y_{t-1} + \varepsilon_t$
Drift term	No	Yes
Expected change	0	c
Systematic trend	No	Yes, on average
If $c > 0$	—	Upward tendency
If $c < 0$	—	Downward tendency

Simulate the random walk model

The random walk (RW) model is also a basic time series model. It is the cumulative sum (or integration) of a mean zero white noise (WN) series, such that the first difference series of a RW is a WN series.

Note for reference that the RW model is an **ARIMA(0, 1, 0)** model, in which the middle entry of 1 indicates that the model's order of integration is 1.

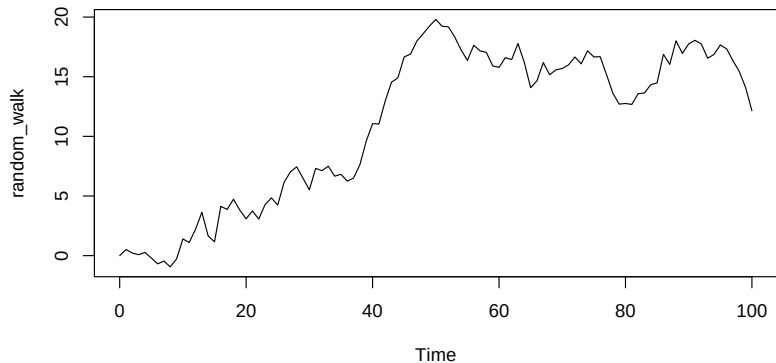
The `arima.sim()` function can be used to simulate data from the RW by including the `model = list(order = c(0, 1, 0))` argument. We also need to specify a series length `n`. Finally, you can specify a `sd` for the series (increments), where the default value is 1.

Simulate the random walk model (cont.)

```
# Generate a RW model using arima.sim
random_walk <- arima.sim(model = list(order = c(0, 1, 0)),
                          n = 100)

# Plot random_walk
ts.plot(random_walk)
```

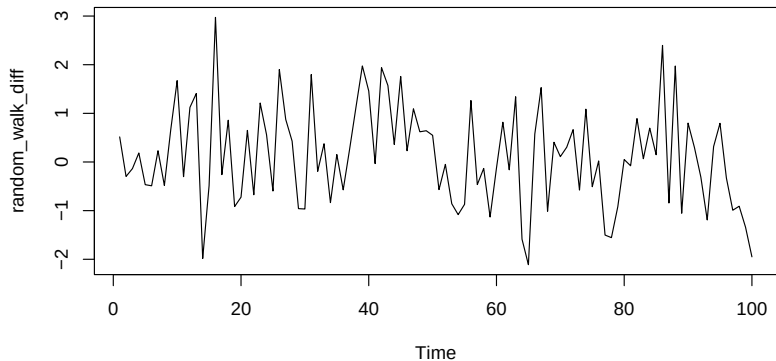
Simulate the random walk model (cont.)



Simulate the random walk model (cont.)

```
# Calculate the first difference series  
random_walk_diff <- diff(random_walk)  
  
# Plot random_walk_diff  
  
ts.plot(random_walk_diff)
```

Simulate the random walk model (cont.)



Simulate the random walk model with a drift

A random walk (RW) need not wander about zero, it can have an upward or downward trajectory, i.e., a drift or time trend. This is done by including an intercept in the RW model, which corresponds to the slope of the RW time trend.

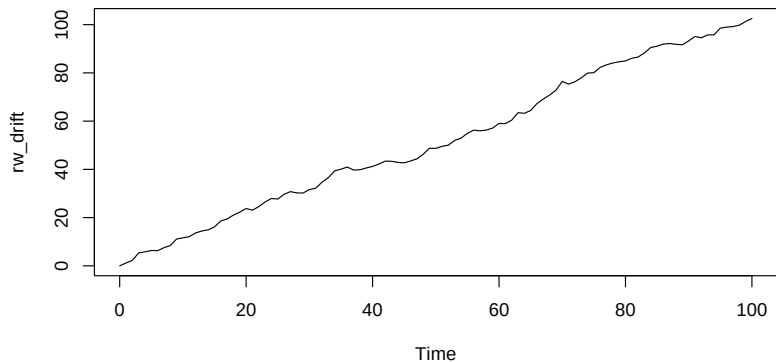
For an alternative formulation, you can take the cumulative sum of a constant mean white noise (WN) series, such that the mean corresponds to the slope of the RW time trend.

To simulate data from the RW model with a drift you again use the `arma.sim()` function with the `model = list(order = c(0, 1, 0))` argument. This time, you should add the additional argument `mean = ...` to specify the drift variable, or the intercept.

Simulate the random walk model with a drift (cont.)

```
# Generate a RW model with a drift using arima.sim  
rw_drift <- arima.sim(model = list(order = c(0, 1, 0)), n =  
  
# Plot rw_drift  
ts.plot(rw_drift)
```

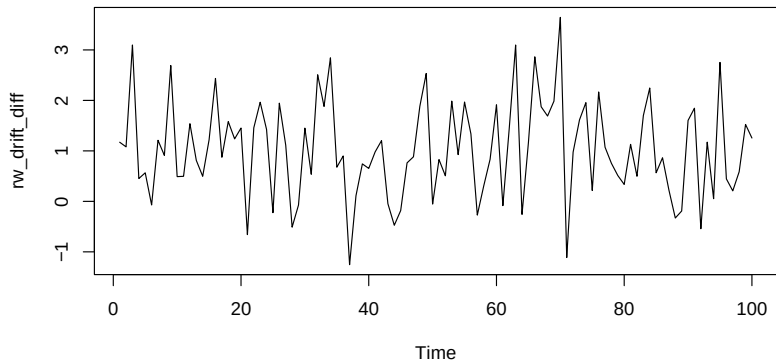
Simulate the random walk model with a drift (cont.)



Simulate the random walk model with a drift (cont.)

```
# Calculate the first difference series  
rw_drift_diff <- diff(rw_drift)  
  
# Plot rw_drift_diff  
  
ts.plot(rw_drift_diff)
```

Simulate the random walk model with a drift (cont.)



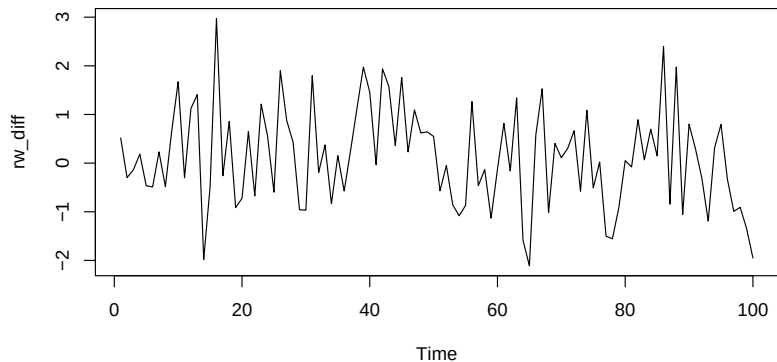
Estimate the random walk model

For a given time series y we can fit the random walk model with a drift by first differencing the data, then fitting the white noise (WN) model to the differenced data using the `arma()` command with the `order = c(0, 0, 0)` argument.

The `arma()` command displays information or output about the fitted model. Under the **Coefficients:** heading is the estimated drift variable, named the `intercept`. Its approximate standard error (or s.e.) is provided directly below it. The variance of the WN part of the model is also estimated under the label σ^2 .

```
# Difference your random_walk data  
rw_diff <- diff(random_walk)  
  
# Plot rw_diff  
ts.plot(rw_diff)
```

Estimate the random walk model (cont.)



Estimate the random walk model (cont.)

```
# Now fit the WN model to the differenced data
model_wn <- arima(x = rw_diff, order = c(0, 0, 0))

# Store the value of the estimated time trend (intercept)
int_wn <- model_wn$coef

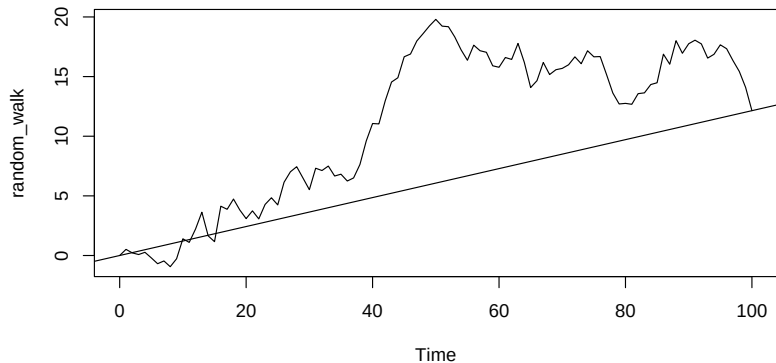
# Plot the original random_walk data

ts.plot(random_walk)

# Use abline(0, ...) to add time trend to the figure

abline(0, int_wn)
```

Estimate the random walk model (cont.)



Section 11

Stationarity

Stationarity

- ▶ Stationary models are parsimonious.
- ▶ Stationary processes have distributional stability over time.

Observed time series:

- ▶ Fluctuate randomly.
- ▶ But behave similarly from one time period to the next.

Weak stationarity

Weak stationary: mean, variance, covariance constant over time.

$Y_1, Y_2, \dots$ a weakly stationary process if:

- ▶ Mean μ of Y_t is same (constant) for all t .
- ▶ Variance σ^2 of Y_t is same (constant) for all t .
- ▶ Covariance of Y_t and Y_s is same (constant) for all $|t - s| = h$, for all h .

$$Cov(Y_2, Y_5) = Cov(Y_7, Y_{10})$$

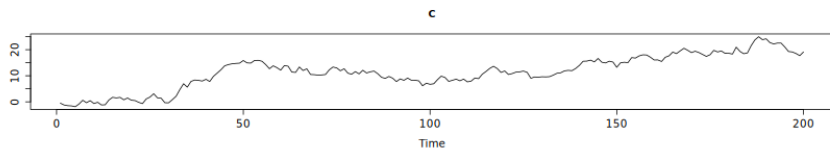
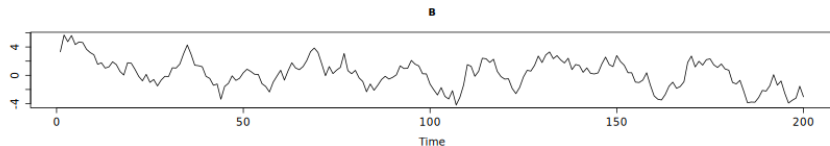
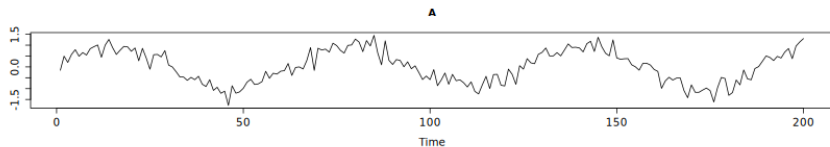
since each pair is separated by three units of time.

Stationary or not?

There are many commonly encountered departures from stationarity, including time trends, periodicity, and a lack of mean reversion.

Which of the three time series plots below appear to be from a stationary model?

Stationary or not? (cont.)



Answer: ✓ B.

Are the white noise model or the random walk model stationary?

The white noise (WN) and random walk (RW) models are very closely related. However, only the RW is always non-stationary, both with and without a drift term. This is a simulation exercise to highlight the differences.

Recall that if we start with a mean zero WN process and compute its running or cumulative sum, the result is a RW process. The `cumsum()` function will make this transformation for you. Similarly, if we create a WN process, but change its mean from zero, and then compute its cumulative sum, the result is a RW process with a drift.

Are the white noise model or the random walk model stationary? (cont.)

```
# Use arima.sim() to generate WN data
white_noise <- arima.sim(model = list(order = c(0, 0, 0)), n = 1000)

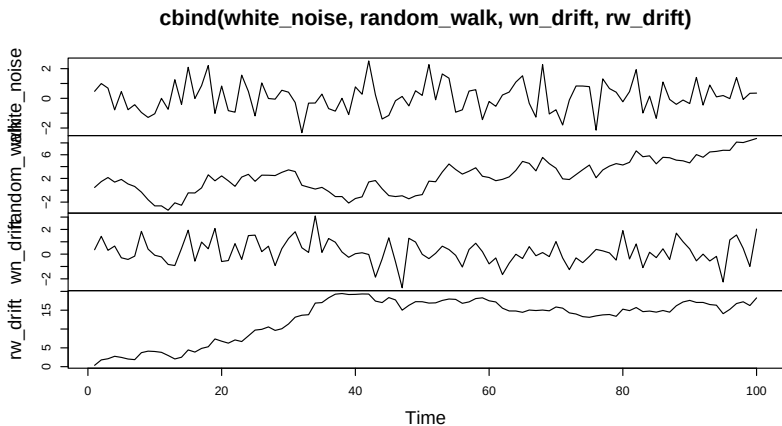
# Use cumsum() to convert your WN data to RW
random_walk <- cumsum(white_noise)

# Use arima.sim() to generate WN drift data
wn_drift <- arima.sim(model = list(order = c(0, 0, 0)), n = 1000)

# Use cumsum() to convert your WN drift data to RW
rw_drift <- cumsum(wn_drift)

# Plot all four data objects
plot.ts(cbind(white_noise, random_walk, wn_drift, rw_drift))
```

Are the white noise model or the random walk model stationary? (cont.)



Section 12

Correlation & ACF

Scatterplot

Asset prices vs. asset returns

The goal of investing is to make a profit. The revenue or loss from investing depends on the amount invested and changes in prices, and high revenue relative to the size of an investment is of central interest. This is what financial asset returns measure, changes in price as a fraction of the initial price over a given time horizon, for example, one business day.

Let's again consider the `eu_stocks` dataset. This dataset reports index values, which we can regard as prices. The indices are not investable assets themselves, but there are many investable financial assets that closely track major market indices, including mutual funds and exchange traded funds.

Correlation analysis and the autocorrelation function (cont.)

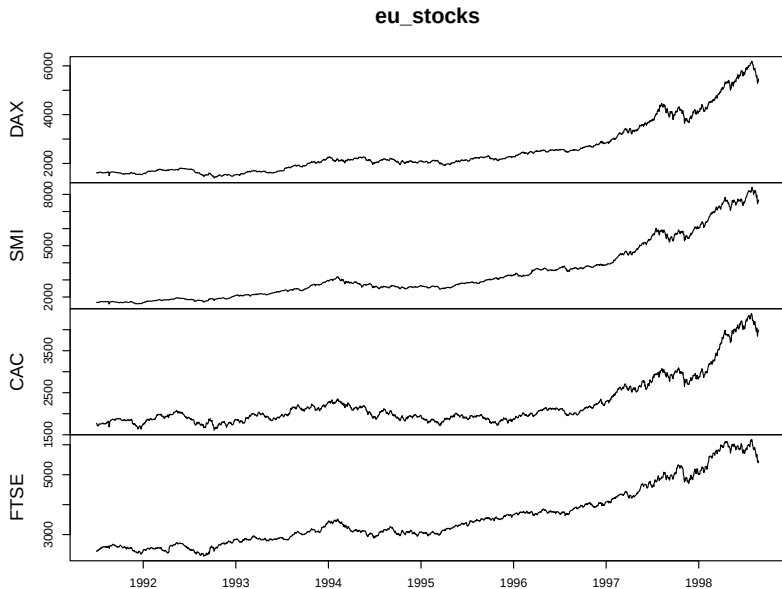
Log returns, also called continuously compounded returns, are also commonly used in financial time series analysis. They are the log of gross returns, or equivalently, the changes (or first differences) in the logarithm of prices.

The change in appearance between daily prices and daily returns is typically substantial, while the difference between daily returns and log returns is usually small. As you'll see later, one advantage of using log returns is that calculating multi-period returns from individual periods is greatly simplified - you just add them together!

In this exercise, you'll further explore the `eu_stocks` dataset, including plotting prices, converting prices to (net) returns, and converting prices to log returns.

```
# Plot eu_stocks  
plot(eu_stocks)
```

Correlation analysis and the autocorrelation function (cont.)



Characteristics of financial time series

Daily financial asset returns typically share many characteristics. Returns over one day are typically small, and their average is close to zero. At the same time, their variances and standard deviations can be relatively large. Over the course of a few years, several very large returns (in magnitude) are typically observed. These relative outliers happen on only a handful of days, but they account for the most substantial movements in asset prices. Because of these extreme returns, the distribution of daily asset returns is not normal, but heavy-tailed, and sometimes skewed. In general, individual stock returns typically have even greater variability and more extreme observations than index returns.

In this exercise, you'll work with the `eu_percentreturns` dataset, which is the percentage returns calculated from your `eu_stocks` data. For each of the four indices contained in your data, you'll calculate the sample mean, variance, and standard deviation.

Characteristics of financial time series (cont.)

Notice that the average daily return is about 0, while the standard deviation is about 1 percentage point. Also apply the `hist()` and `qqnorm()` functions to make histograms and normal quantile plots, respectively, for each of the indices.

```
eu_percentreturns <- returns*100
```

```
# Generate means from eu_percentreturns  
colMeans(eu_percentreturns)
```

```
##           DAX           SMI           CAC           FTSE  
## 0.07052174 0.08609470 0.04979471 0.04637479
```

```
# Use apply to calculate sample variance from eu_percentreturns  
apply(eu_percentreturns, MARGIN = 2, FUN = var)
```

```
##           DAX           SMI           CAC           FTSE  
## 1.0569648 0.8523711 1.2159091 0.6344767
```

Characteristics of financial time series (cont.)

```
# Use apply to calculate standard deviation from eu_percentr  
apply(eu_percentreturns, MARGIN = 2, FUN = sd)
```

```
##           DAX           SMI           CAC           FTSE  
## 1.0280879 0.9232394 1.1026827 0.7965405
```

Plotting pairs of data

Time series data is often presented in a time series plot. For example, the index values from the `eu_stocks` dataset are shown in the adjoining figure. Recall, `eu_stocks` contains daily closing prices from 1991-1998 for the major stock indices in Germany (DAX), Switzerland (SMI), France (CAC), and the UK (FTSE).

It is also useful to examine the bivariate relationship between pairs of time series. In this exercise we will consider the contemporaneous relationship, that is matching observations that occur at the same time, between pairs of index values as well as their log returns. The `plot(a, b)` function will produce a scatterplot when two time series names `a` and `b` are given as input.

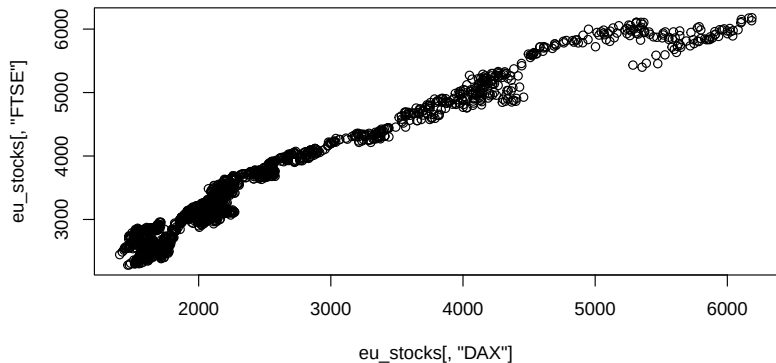
To simultaneously make scatterplots for all pairs of several assets the `pairs()` function can be applied to produce a scatterplot matrix. When shared time trends are present in prices or index values it is common to instead compare their returns or log returns.

Plotting pairs of data (cont.)

In this exercise, you'll practice these skills on the `eu_stocks` data. Because the DAX and FTSE returns have similar time coverage, you can easily make a scatterplot of these indices. Note that the normal distribution has elliptical contours of equal probability, and pairs of data drawn from the multivariate normal distribution form a roughly elliptically shaped point cloud. Do any of the pairs in the scatterplot matrices exhibit this pattern, before or after log transformation?

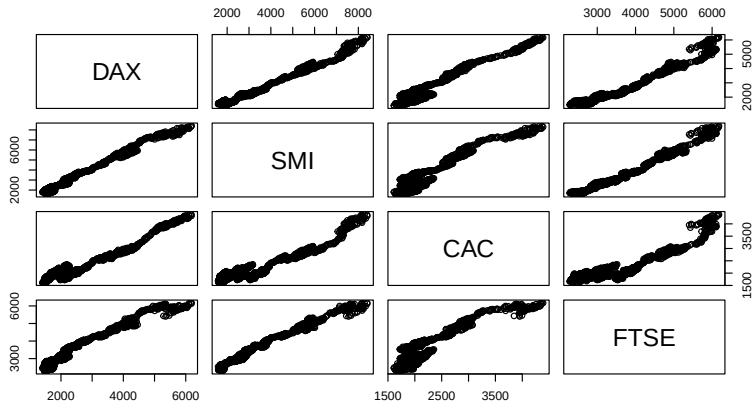
```
# Make a scatterplot of DAX and FTSE  
plot(eu_stocks[, "DAX"], eu_stocks[, "FTSE"])
```

Plotting pairs of data (cont.)



```
# Make a scatterplot matrix of eu_stocks  
pairs(eu_stocks)
```

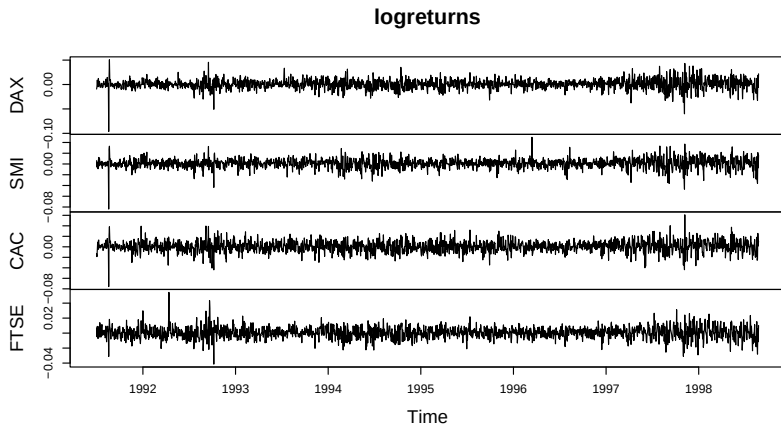
Plotting pairs of data (cont.)



Plotting pairs of data (cont.)

```
# Convert eu_stocks to log returns  
logreturns <- diff(log(eu_stocks))  
  
# Plot logreturns  
plot(logreturns)
```

Plotting pairs of data (cont.)

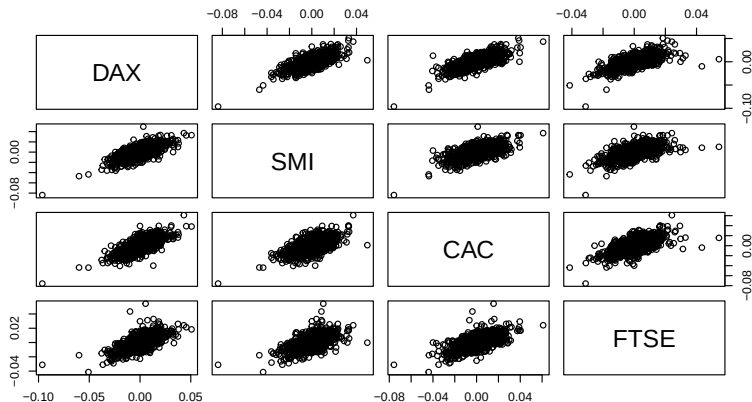


Plotting pairs of data (cont.)

```
# Make a scatterplot matrix of logreturns
```

```
pairs(logreturns)
```

Plotting pairs of data (cont.)



Section 13

Covariance and Correlation

Calculating sample covariances and correlations

Sample covariances measure the strength of the linear relationship between matched pairs of variables. The `cov()` function can be used to calculate covariances for a pair of variables, or a covariance matrix when a matrix containing several variables is given as input. For the latter case, the matrix is symmetric with covariances between variables on the off-diagonal and variances of the variables along the diagonal. On the right you can see the scatterplot matrix of your `logreturns` data.

Covariances are very important throughout finance, but they are not scale free and they can be difficult to directly interpret. Correlation is the standardized version of covariance that ranges in value from -1 to 1, where values close to 1 in magnitude indicate a strong linear relationship between pairs of variables. The `cor()` function can be applied to both pairs of variables as well as a matrix containing several variables, and the output is interpreted analogously.

Calculating sample covariances and correlations (cont.)

```
# Use cov() with DAX_logreturns and FTSE_logreturns  
cov(logreturns[, "DAX"], logreturns[, "FTSE"])
```

```
## [1] 5.241794e-05
```

```
# Use cov() with logreturns  
cov(logreturns)
```

```
##           DAX           SMI           CAC           FTSE  
## DAX  1.061072e-04  6.699564e-05  8.345130e-05  5.241794e-05  
## SMI  6.699564e-05  8.556317e-05  6.285881e-05  4.304517e-05  
## CAC  8.345130e-05  6.285881e-05  1.216802e-04  5.693174e-05  
## FTSE 5.241794e-05  4.304517e-05  5.693174e-05  6.332543e-05
```

```
# Use cor() with DAX_logreturns and FTSE_logreturns  
cov(logreturns[, "DAX"], logreturns[, "FTSE"])
```

Calculating sample covariances and correlations (cont.)

```
## [1] 5.241794e-05
```

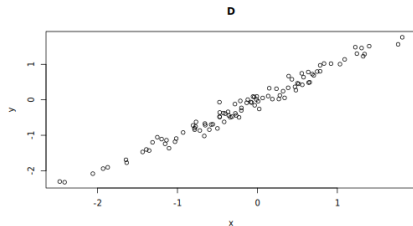
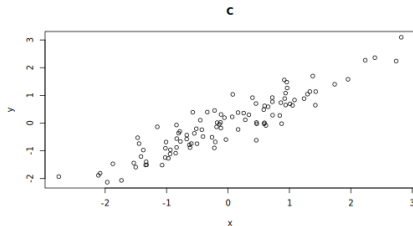
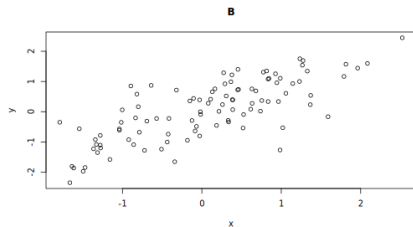
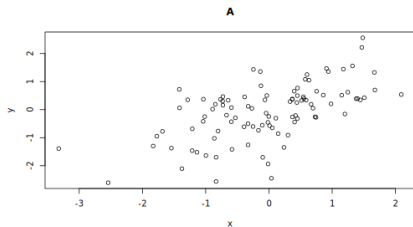
```
# Use cor() with logreturns  
cor(logreturns)
```

```
##           DAX           SMI           CAC           FTSE  
## DAX  1.0000000  0.7031219  0.7344304  0.6394674  
## SMI  0.7031219  1.0000000  0.6160454  0.5847791  
## CAC  0.7344304  0.6160454  1.0000000  0.6485679  
## FTSE 0.6394674  0.5847791  0.6485679  1.0000000
```

Calculating sample covariances and correlations (cont.)

Guess the correlation coefficient

Which scatterplot has a sample correlation of approximately 0.90?



Section 14

Autocorrelation

Calculating autocorrelations

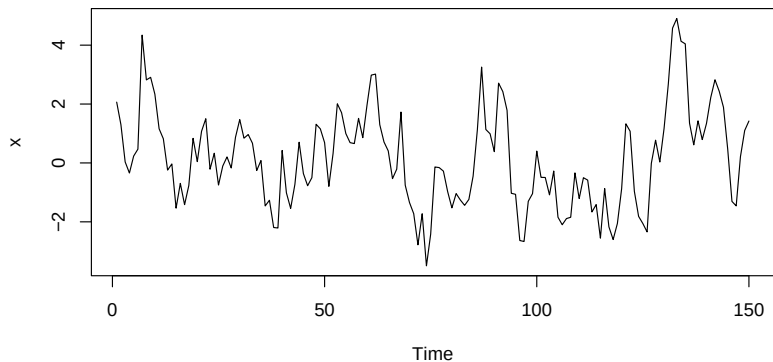
Autocorrelations or lagged correlations are used to assess whether a time series is dependent on its past. For a time series x of length n we consider the $n-1$ pairs of observations one time unit apart. The first such pair is $(x[2], x[1])$, and the next is $(x[3], x[2])$. Each such pair is of the form $(x[t], x[t-1])$ where t is the observation index, which we vary from 2 to n in this case. The lag-1 autocorrelation of x can be estimated as the sample correlation of these $(x[t], x[t-1])$ pairs.

In general, we can manually create these pairs of observations. First, create two vectors, x_t0 and x_t1 , each with length $n-1$, such that the rows correspond to $(x[t], x[t-1])$ pairs. Then apply the `cor()` function to estimate the lag-1 autocorrelation.

Luckily, the `acf()` command provides a shortcut. Applying `acf(..., lag.max = 1, plot = FALSE)` to a series x automatically calculates the lag-1 autocorrelation.

Finally, note that the two estimates differ slightly as they use slightly different scalings in their calculation of sample covariance, $1/(n-1)$ versus $1/n$. Although the latter would provide a biased estimate, it is preferred in time series analysis, and the resulting autocorrelation estimates only differ by a factor of $(n-1)/n$.

Calculating autocorrelations (cont.)



Calculating autocorrelations (cont.)

```
n <- length(x)

# Define x_t0 as x[-1]
x_t0 <- x[-1]

# Define x_t1 as x[-n]
x_t1 <- x[-n]

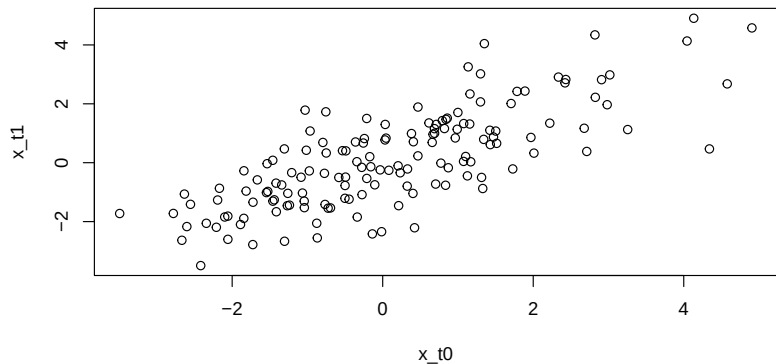
# Confirm that x_t0 and x_t1 are (x[t], x[t-1]) pairs
head(cbind(x_t0, x_t1))
```

Calculating autocorrelations (cont.)

```
##           x_t0           x_t1
## [1,]  1.2996380  2.0655438
## [2,]  0.0335780  1.2996380
## [3,] -0.3425806  0.0335780
## [4,]  0.2325613 -0.3425806
## [5,]  0.4681201  0.2325613
## [6,]  4.3411156  0.4681201
```

```
# Plot x_t0 and x_t1
plot(x_t0, x_t1)
```

Calculating autocorrelations (cont.)



```
# View the correlation between x_t0 and x_t1  
cor(x_t0, x_t1)
```

Calculating autocorrelations (cont.)

```
## [1] 0.7630314
```

```
# Use acf with x
```

```
acf(x, lag.max = 1, plot = FALSE)
```

```
##
```

```
## Autocorrelations of series 'x', by lag
```

```
##
```

```
##      0      1
```

```
## 1.000 0.758
```

```
# Confirm that difference factor is (n-1)/n
```

```
cor(x_t1, x_t0) * (n-1)/n
```

```
## [1] 0.7579445
```

The autocorrelation function

Autocorrelations can be estimated at many lags to better assess how a time series relates to its past. We are typically most interested in how a series relates to its most recent past.

The

```
acf(..., lag.max = ..., plot = FALSE)
```

function will estimate all autocorrelations from 0, 1, 2,..., up to the value specified by the argument `lag.max`. In the previous exercise, you focused on the lag-1 autocorrelation by setting the `lag.max` argument to 1.

```
# Generate ACF estimates for x up to lag-10  
acf(x, lag.max = 10, plot = FALSE)
```

The autocorrelation function (cont.)

```
##  
## Autocorrelations of series 'x', by lag  
##  
##      0      1      2      3      4      5      6      7      8  
## 1.000 0.758 0.537 0.345 0.227 0.198 0.140 0.114 0.124 0.1
```

Visualizing the autocorrelation function

Estimating the autocorrelation function (ACF) at many lags allows us to assess how a time series x relates to its past. The numeric estimates are important for detailed calculations, but it is also useful to visualize the ACF as a function of the lag.

In fact, the `acf()` command produces a figure by default. It also makes a default choice for `lag.max`, the maximum number of lags to be displayed.

Three time series x , y , and z have been loaded into your R environment and are plotted on the right. The time series x shows strong persistence, meaning the current value is closely related to those that precede it. The time series y shows a periodic pattern with a cycle length of approximately four observations, meaning the current value is relatively close to the observation four before it. The time series z does not exhibit any clear pattern.

Visualizing the autocorrelation function (cont.)

In this exercise, you'll plot an estimated autocorrelation function for each time series. In the plots produced by `acf()`, the lag for each autocorrelation estimate is denoted on the horizontal axis and each autocorrelation estimate is indicated by the height of the vertical bars. Recall that the ACF at lag-0 is always 1.

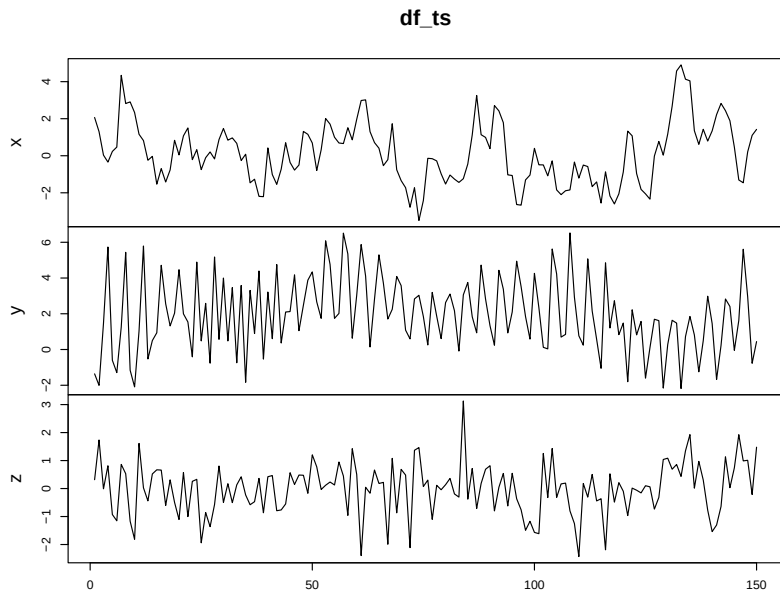
Finally, each ACF figure includes a pair of blue, horizontal, dashed lines representing lag-wise 95% confidence intervals centered at zero. These are used for determining the statistical significance of an individual autocorrelation estimate at a given lag versus a null value of zero, i.e., no autocorrelation at that lag.

```
df <- read.table("x_y_z_three_columns.txt", header = T)

df_ts <- ts(data = df, start = c(1, 1), frequency = 1)

plot(df_ts)
```

Visualizing the autocorrelation function (cont.)



Section 15

Autoregressive Models

Autoregressive Models

The Autoregressive (AR) recursion:

$$Today = Constant + Slope * Yesterday + Noise$$

Mean centered version:

$$(Today - Mean) = Slope * (Yesterday - Mean) + Noise$$

More formally:

$$Y_t - \mu = \phi(Y_{t-1} - \mu) + \epsilon_t$$

Where ϵ_t is mean zero white noise (WN).

- ▶ The mean μ .
- ▶ The slope ϕ .

- ▶ The WN variance σ^2 .

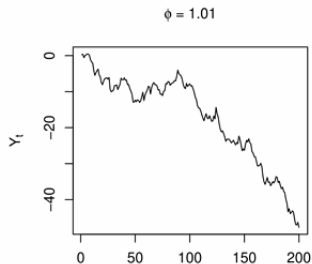
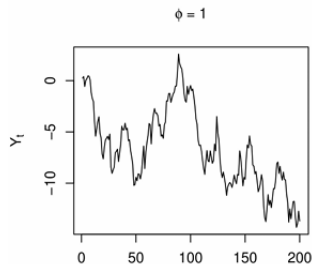
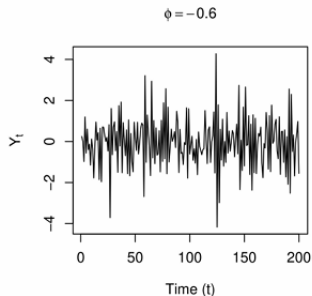
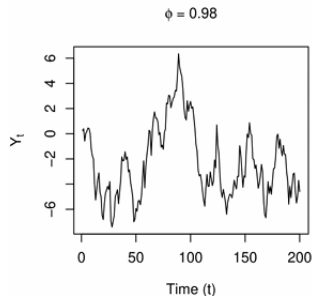
AR processes - I

$$Y_t - \mu = \phi(Y_{t-1} - \mu) + \epsilon_t$$

- ▶ If slope $\phi = 0$ then: $Y_t = \mu + \epsilon_t$ and Y_t is white noise: (μ, σ_ϵ^2) .
- ▶ If slope $\phi \neq 0$ then: Y_t depends on both ϵ_t and Y_{t-1} and the process Y_t is autocorrelated.
- ▶ Large values of ϕ lead to greater autocorrelation.
- ▶ Negative values of ϕ result in oscillatory time series.

AR example

Autoregressive Models (cont.)



Simulate the autoregressive model

The autoregressive (AR) model is arguably the most widely used time series model. It shares the very familiar interpretation of a simple linear regression, but here each observation is regressed on the previous observation. The AR model also includes the white noise (WN) and random walk (RW) models examined in earlier chapters as special cases.

The versatile `arima.sim()` function used in previous chapters can also be used to simulate data from an AR model by setting the `model` argument equal to `list(ar = phi)`, in which `phi` is a slope parameter from the interval $(-1, 1)$. We also need to specify a series length `n`.

Simulate the autoregressive model (cont.)

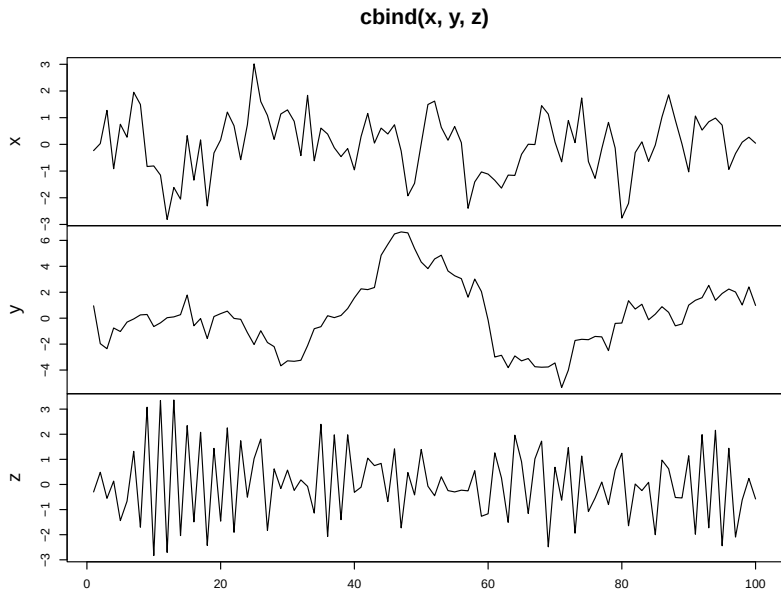
```
# Simulate an AR model with 0.5 slope
x <- arima.sim(model = list(ar = 0.5), n = 100)

# Simulate an AR model with 0.9 slope
y <- arima.sim(model = list(ar = 0.9), n = 100)

# Simulate an AR model with -0.75 slope
z <- arima.sim(model = list(ar = -0.75), n = 100)

# Plot your simulated data
plot.ts(cbind(x, y, z))
```

Simulate the autoregressive model (cont.)

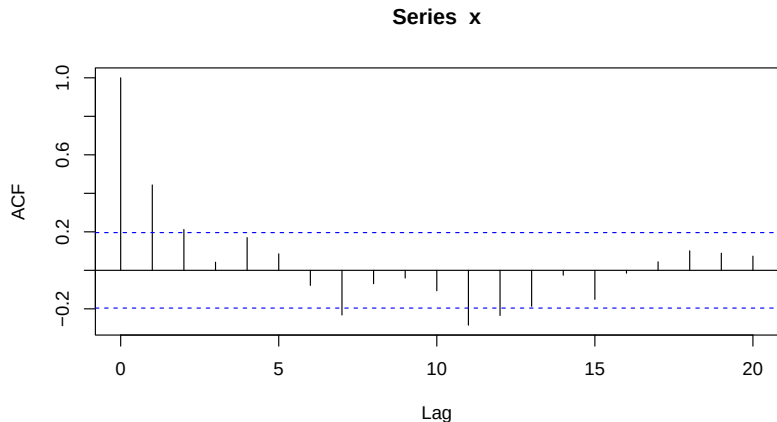


Estimate the autocorrelation function (ACF) for an autoregression

What if you need to estimate the autocorrelation function from your data? To do so, you'll need the `acf()` command, which estimates autocorrelation by exploring lags in your data. By default, this command generates a plot of the relationship between the current observation and lags extending backwards.

```
# Calculate the ACF for x  
acf(x, plot = TRUE)
```

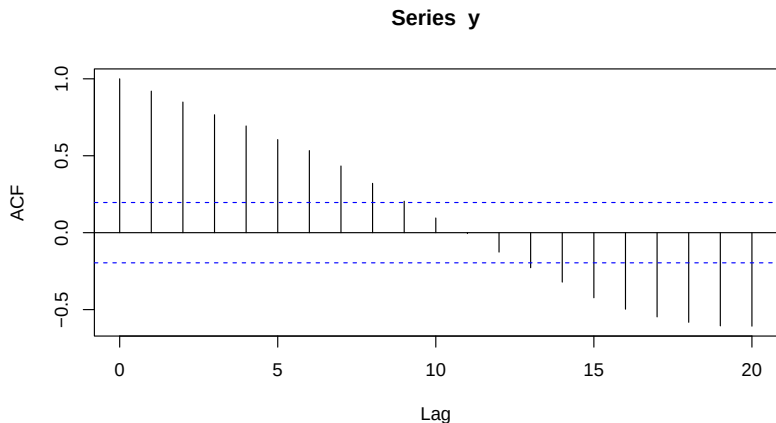
Estimate the autocorrelation function (ACF) for an autoregression (cont.)



Estimate the autocorrelation function (ACF) for an autoregression (cont.)

```
# Calculate the ACF for y  
acf(y, plot = TRUE)
```

Estimate the autocorrelation function (ACF) for an autoregression (cont.)

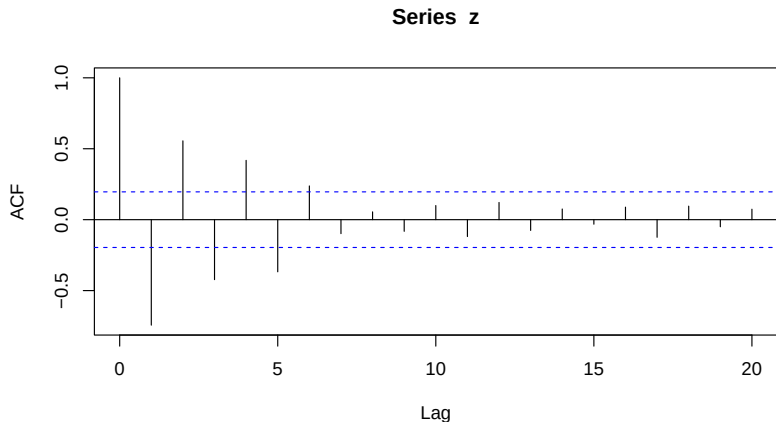


Estimate the autocorrelation function (ACF) for an autoregression (cont.)

```
# Calculate the ACF for z
```

```
acf(z, plot = TRUE)
```

Estimate the autocorrelation function (ACF) for an autoregression (cont.)



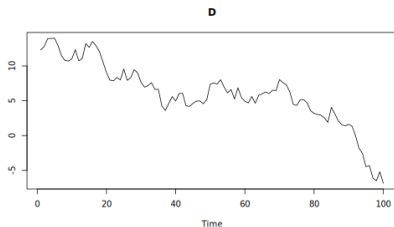
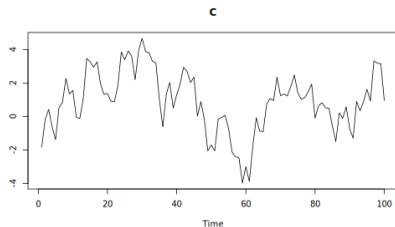
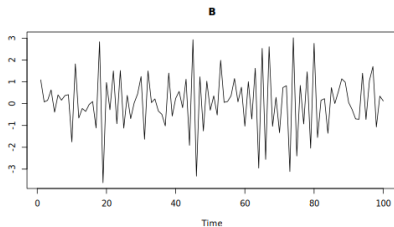
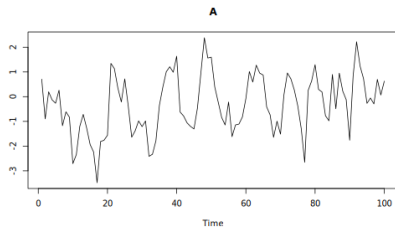
Estimate the autocorrelation function (ACF) for an autoregression (cont.)

The plots generated by the `acf()` command provide useful information about each lag of your time series. The first series `x` has positive autocorrelation for the first couple lags, but they quickly approach zero. The second series `y` has positive autocorrelation for many lags, but they also decay to zero. The last series `z` has an alternating pattern, as does its autocorrelation function (ACF), but its ACF still quickly decays to zero in magnitude.

Autoregressive processes can exhibit varying levels of persistence as well as anti-persistence or oscillatory behavior. Persistence is defined by a high correlation between an observation and its lag, while anti-persistence is defined by a large amount of variation between an observation and its lag.

The four plots below show varying degrees of persistence and anti-persistence. Which series exhibits the greatest persistence?

Persistence and anti-persistence (cont.)



Answer: ✓ D.

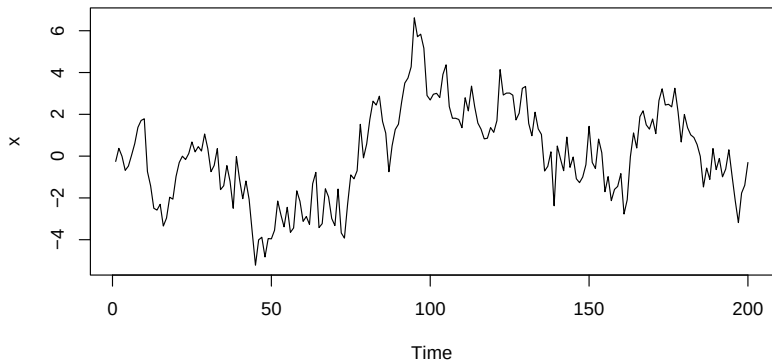
Compare the random walk (RW) and autoregressive (AR) models

The random walk (RW) model is a special case of the autoregressive (AR) model, in which the slope parameter is equal to 1. Recall from previous chapters that the RW model is not stationary and exhibits very strong persistence. Its sample autocovariance function (ACF) also decays to zero very slowly, meaning past values have a long lasting impact on current values.

The stationary AR model has a slope parameter between -1 and 1. The AR model exhibits higher persistence when its slope parameter is closer to 1, but the process reverts to its mean fairly quickly. Its sample ACF also decays to zero at a quick (geometric) rate, indicating that values far in the past have little impact on future values of the process.

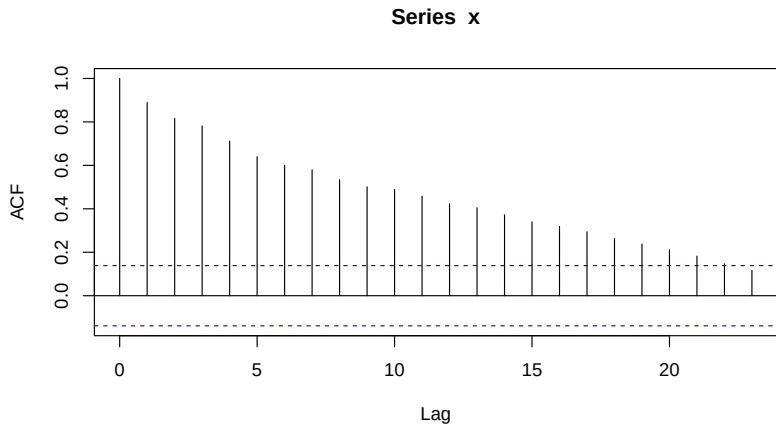
```
# Simulate and plot AR model with slope 0.9  
x <- arima.sim(model = list(ar = 0.9), n = 200)  
ts.plot(x)
```

Compare the random walk (RW) and autoregressive (AR) models (cont.)



Compare the random walk (RW) and autoregressive (AR) models (cont.)

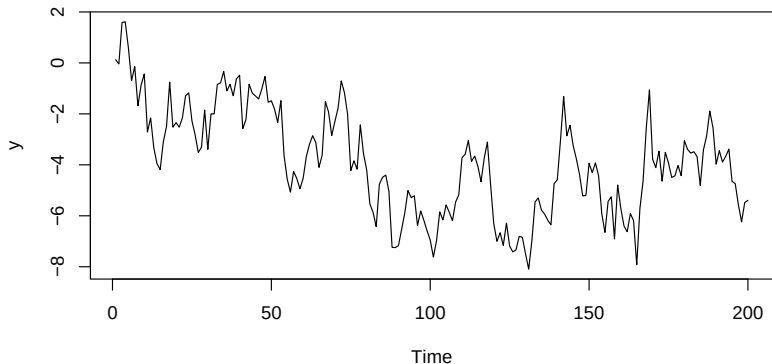
```
acf(x, plot = TRUE)
```



Compare the random walk (RW) and autoregressive (AR) models (cont.)

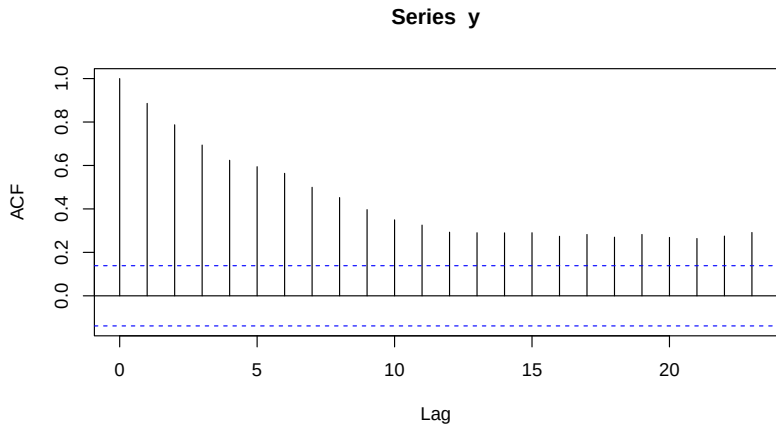
```
# Simulate and plot AR model with slope 0.98  
y <- arima.sim(model = list(ar = 0.98), n = 200)  
ts.plot(y)
```

Compare the random walk (RW) and autoregressive (AR) models (cont.)



Compare the random walk (RW) and autoregressive (AR) models (cont.)

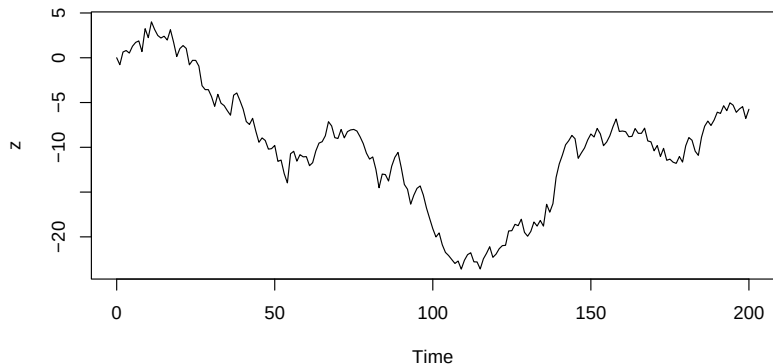
```
acf(y, plot = TRUE)
```



Compare the random walk (RW) and autoregressive (AR) models (cont.)

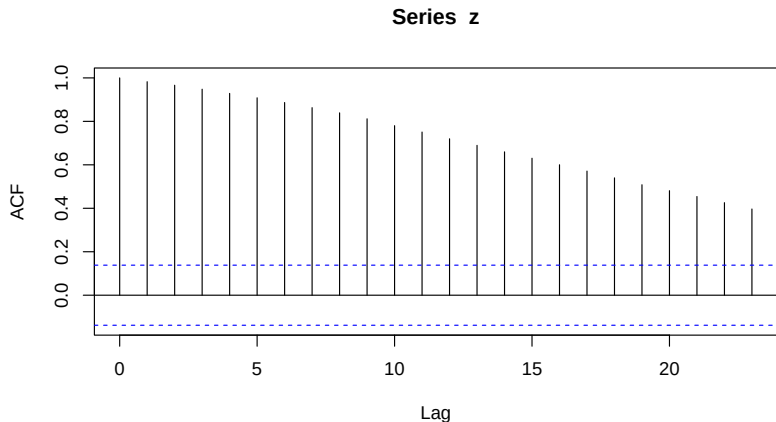
```
# Simulate and plot RW model  
z <- arima.sim(model = list(order = c(0, 1, 0)), n = 200)  
ts.plot(z)
```

Compare the random walk (RW) and autoregressive (AR) models (cont.)



Compare the random walk (RW) and autoregressive (AR) models (cont.)

`acf(z)`



- ▶ AR fitted values:

$$\hat{Y}_t = \hat{\mu} + \hat{\phi}(Y_{t-1} - \hat{\mu})$$

- ▶ Residuals:

$$\hat{\epsilon}_t = Y_t - \hat{Y}_t$$

- ▶ Fitted value

$$\hat{Y}_t = Y_t - \hat{\epsilon}_t$$

Estimate the autoregressive (AR) model

For a given time series x we can fit the autoregressive (AR) model using the `arima()` command and setting order equal to `c(1, 0, 0)`. Note for reference that an AR model is an **ARIMA(1, 0, 0)** model. In this exercise, you'll explore additional qualities of the AR model by practicing the `arima()` command on a simulated time series x as well as the `AirPassengers` data. This command allows you to identify the estimated slope (`ar1`), mean (`intercept`), and innovation variance (`sigma^2`) of the model.

```
# Fit the AR model to AirPassengers  
AR <- arima(AirPassengers, order = c(1, 0, 0))  
print(AR)
```

Estimate the autoregressive (AR) model (cont.)

```
##  
## Call:  
## arima(x = AirPassengers, order = c(1, 0, 0))  
##  
## Coefficients:  
##          ar1  intercept  
##       0.9646   278.4649  
## s.e.  0.0214    67.1141  
##  
## sigma^2 estimated as 1119:  log likelihood = -711.09,  ai
```

Estimate the autoregressive (AR) model (cont.)

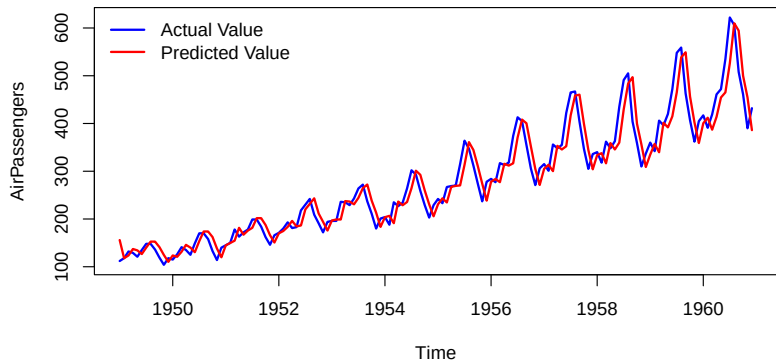
Run the following commands to plot the series and fitted values

```
ts.plot(AirPassengers, col = "blue", lwd = 2)
AR_fitted <- AirPassengers - residuals(AR)
points(AR_fitted, type = "l", col = "red", lty = 1, lwd = 2)

labels <- c("Actual Value", "Predicted Value")
colors <- c("blue", "red")

legend("topleft", labels, col = colors, lty = 1, lwd = 2, bty = "n")
```

Estimate the autoregressive (AR) model (cont.)



Estimate the autoregressive (AR) model (cont.)

By fitting an AR model to the AirPassengers data, you've successfully modeled the data in a reproducible fashion. This allows you to predict future observations based on your AR_fitted data.

Simple forecasts from an estimated AR model

Now that you've modeled your data using the `arima()` command, you are ready to make simple forecasts based on your model. The `predict()` function can be used to make forecasts from an estimated AR model. In the object generated by your `predict()` command, the `$pred` value is the forecast, and the `$se` value is the standard error for the forecast.

To make predictions for several periods beyond the last observations, you can use the `n.ahead` argument in your `predict()` command. This argument establishes the forecast horizon (`h`), or the number of periods being forecast. The forecasts are made recursively from 1 to `h`-steps ahead from the end of the observed time series.

In this exercise, you'll make simple forecasts using an AR model applied to the Nile data, which records annual observations of the flow of the River Nile from 1871 to 1970.

Simple forecasts from an estimated AR model (cont.)

```
# Fit an AR model to Nile
```

```
AR_fit <- arima(Nile, order = c(1, 0, 0))
```

```
print(AR_fit)
```

```
##
```

```
## Call:
```

```
## arima(x = Nile, order = c(1, 0, 0))
```

```
##
```

```
## Coefficients:
```

```
##           ar1  intercept
```

```
##          0.5063    919.5685
```

```
## s.e.    0.0867     29.1410
```

```
##
```

```
## sigma^2 estimated as 21125:  log likelihood = -639.95, a
```

Simple forecasts from an estimated AR model (cont.)

```
# Use predict() to make a 1-step forecast
```

```
predict_AR <- predict(AR_fit)
```

```
# Obtain the 1-step forecast using $pred[1]
```

```
predict_AR$pred[1]
```

```
## [1] 828.6576
```

```
# Use predict to make 1-step through 10-step forecasts
```

```
predict(AR_fit, n.ahead = 10)
```

```
## $pred
```

```
## Time Series:
```

```
## Start = 1971
```

```
## End = 1980
```

```
## Frequency = 1
```

```
## [1] 828.6576 873.5426 896.2668 907.7715 913.5960 916.544
```

Simple forecasts from an estimated AR model (cont.)

```
## [9] 919.1762 919.3699
##
## $se
## Time Series:
## Start = 1971
## End = 1980
## Frequency = 1
## [1] 145.3439 162.9092 167.1145 168.1754 168.4463 168.515
## [9] 168.5391 168.5394
```

Simple forecasts from an estimated AR model (cont.)

```
# Run to plot the Nile series plus the forecast and 95% pred  
ts.plot(Nile, xlim = c(1871, 1980), col = "blue")  
AR_forecast <- predict(AR_fit, n.ahead = 10)$pred  
AR_forecast_se <- predict(AR_fit, n.ahead = 10)$se  
points(AR_forecast,  
       type = "l",  
       col = "red")  
points(AR_forecast - 2*AR_forecast_se,  
       type = "l",  
       col = "magenta",  
       lty = 2)  
points(AR_forecast + 2*AR_forecast_se,  
       type = "l",  
       col = "magenta",  
       lty = 2)  
  
labels <- c("Actual Value", "10 Years Forecasted Value",
```

Section 16

Moving Average Models

A Simple Moving Average Model

The simple moving average (MA) model:

$$Today = Mean + Noise + Slope * (Yesterday's Noise)$$

More formally:

$$Y_t = \mu + \epsilon_t + \theta\epsilon_{t-1}$$

where ϵ_t is mean zero white noise (WN).

Three parameters:

- ▶ The mean μ
- ▶ The slope θ
- ▶ The WN variance σ^2
- ▶ If slope θ is zero then:

A Simple Moving Average Model (cont.)

$$Y_t = \mu + \epsilon_t$$

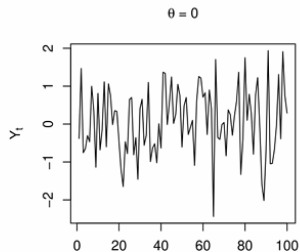
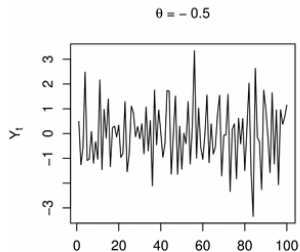
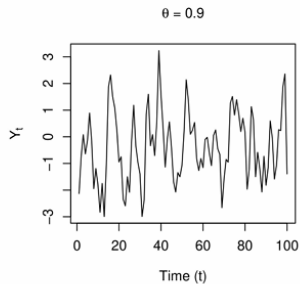
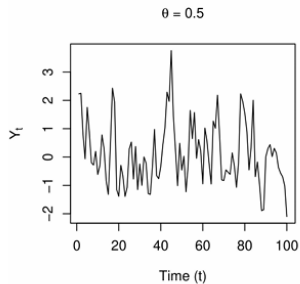
And Y_t is White Noise (μ, σ_ϵ^2)

- ▶ If slope θ is not zero then Y_t depends on both ϵ_t and ϵ_{t-1}

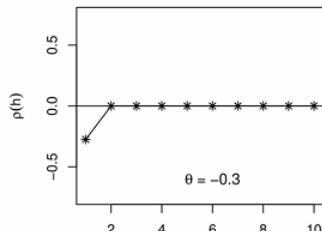
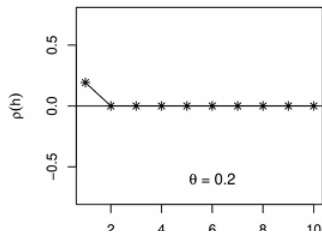
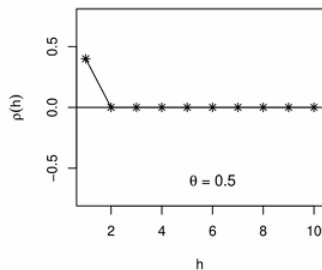
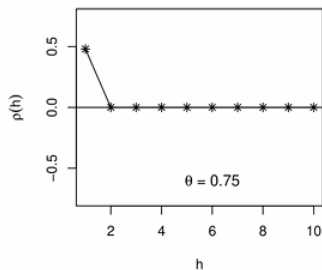
And the process Y_t is autocorrelated

- ▶ Large values of θ lead to greater autocorrelation
- ▶ Negative values of θ result in oscillatory time series

MA Examples



Autocorrelations



Simulate the simple moving average model

The simple moving average (MA) model is a parsimonious time series model used to account for very short-run autocorrelation. It does have a regression like form, but here each observation is regressed on the previous innovation, which is not actually observed. Like the autoregressive (AR) model, the MA model includes the white noise (WN) model as special case.

As with previous models, the MA model can be simulated using the `arma.sim()` command by setting the `model` argument to `list(ma = theta)`, where `theta` is a slope parameter from the interval $(-1, 1)$. Once again, you also need to specify the series length using the `n` argument.

Simulate the simple moving average model (cont.)

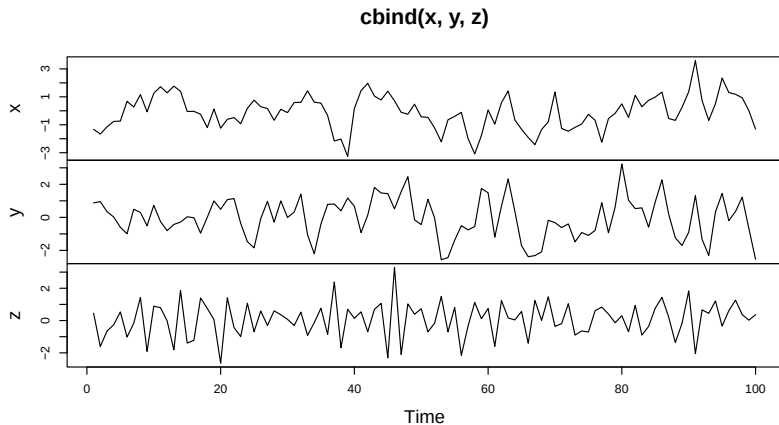
```
# Generate MA model with slope 0.5
x <- arima.sim(model = list(ma = 0.5), n = 100)

# Generate MA model with slope 0.9
y <- arima.sim(model = list(ma = 0.9), n = 100)

# Generate MA model with slope -0.5
z <- arima.sim(model = list(ma = -0.5), n = 100)

# Plot all three models together
plot.ts(cbind(x, y, z), col = 1:3)
```

Simulate the simple moving average model (cont.)



Simulate the simple moving average model (cont.)

Note that there is some very short-run persistence for the positive slope values (x and y), and the series has a tendency to alternate when the slope value is negative (z).

Estimate the autocorrelation function (ACF) for a moving average

Now that you've simulated some MA data using the `arma.sim()` command, you may want to estimate the autocorrelation functions (ACF) for your data. As in the previous chapter, you can use the `acf()` command to generate plots of the autocorrelation in your MA data.

In this exercise, you'll use `acf()` to estimate the ACF for three simulated MA series, `x`, `y`, and `z`. These series have slope parameters of 0.4, 0.9, and -0.75, respectively, and are shown in the figure on the right.

Estimate the autocorrelation function (ACF) for a moving average (cont.)

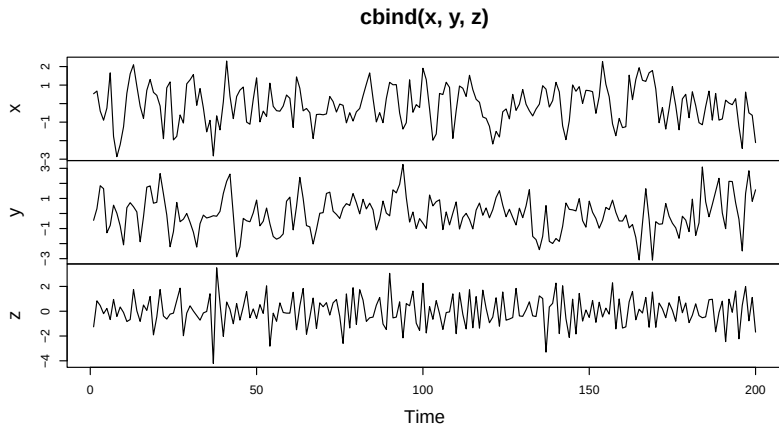
```
x <- arima.sim(model = list(ma = 0.4), n = 200)

# Generate MA model with slope 0.9
y <- arima.sim(model = list(ma = 0.9), n = 200)

# Generate MA model with slope -0.5
z <- arima.sim(model = list(ma = -0.75), n = 200)

# Plot all three models together
plot.ts(cbind(x, y, z), col = 1:3)
```

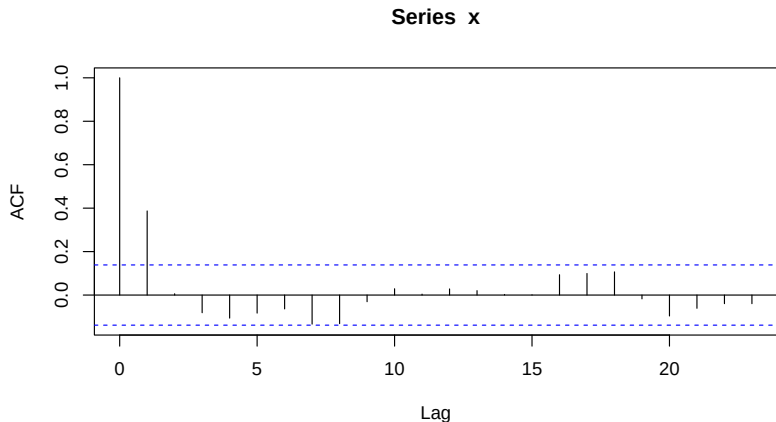
Estimate the autocorrelation function (ACF) for a moving average (cont.)



Estimate the autocorrelation function (ACF) for a moving average (cont.)

```
# Calculate ACF for x  
acf(x, plot = TRUE)
```

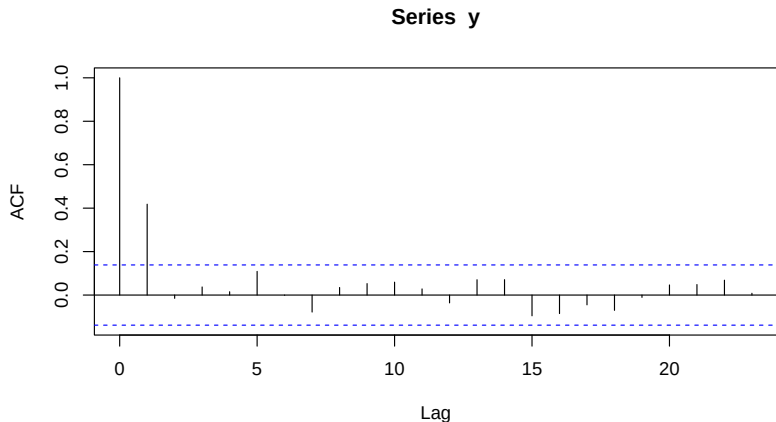
Estimate the autocorrelation function (ACF) for a moving average (cont.)



Estimate the autocorrelation function (ACF) for a moving average (cont.)

```
# Calculate ACF for y  
acf(y, plot = TRUE)
```

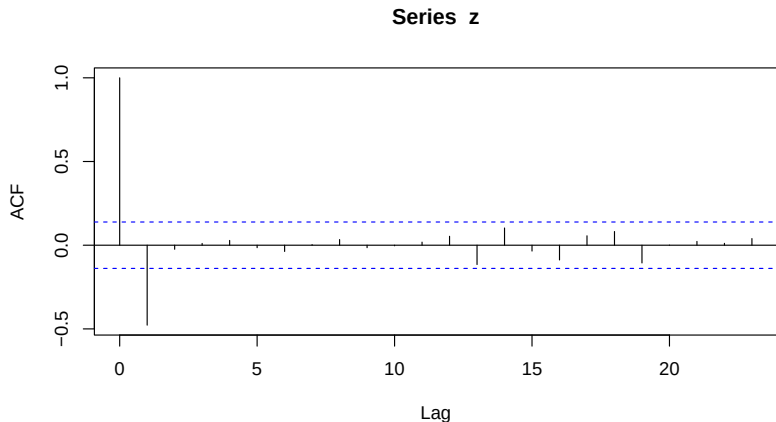
Estimate the autocorrelation function (ACF) for a moving average (cont.)



Estimate the autocorrelation function (ACF) for a moving average (cont.)

```
# Calculate ACF for z  
acf(z, plot = TRUE)
```

Estimate the autocorrelation function (ACF) for a moving average (cont.)



Estimate the autocorrelation function (ACF) for a moving average (cont.)

As you can see from your ACF plots, the series x has positive sample autocorrelation at the first lag, but it is approximately zero at other lags. The series y has a larger sample autocorrelation at its first lag, but it is also approximately zero for the others. The series z has an alternating pattern, and its sample autocorrelation is negative at the first lag. However, similar to the others, it is approximately zero for all higher lags.

Section 17

MA Estimation & Forecasting

- ▶ MA fitted values:

$$\widehat{Today} = \widehat{Mean} + \widehat{Slope} \times \widehat{Yesterday's\ Noise}$$

$$\hat{Y}_t = \hat{\mu} + \hat{\theta}\hat{\epsilon}_{t-1}$$

- ▶ Residuals

$$\hat{\epsilon}_t = Y_t - \hat{Y}_t$$

Estimate the simple moving average model

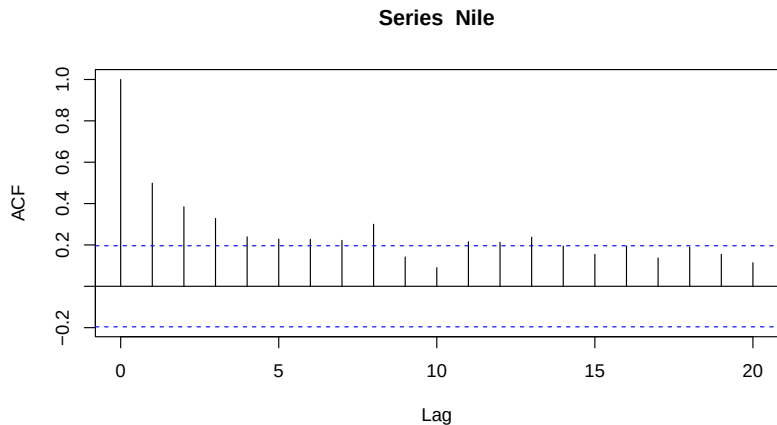
Now that you've simulated some MA models and calculated the ACF from these models, your next step is to fit the simple moving average (MA) model to some data using the `arima()` command. For a given time series `x` we can fit the simple moving average (MA) model using `arima(..., order = c(0, 0, 1))`. Note for reference that an MA model is an **ARIMA(0, 0, 1)** model.

In this exercise, you'll practice using a preloaded time series (`x`, shown in the plot on the right) as well as the Nile dataset used in earlier chapters.

```
#ACF plot of Nile
```

```
acf(Nile, plot = T)
```

Estimate the simple moving average model (cont.)



Estimate the simple moving average model (cont.)

```
# Fit the MA model to Nile
```

```
MA <- arima(Nile, order = c(0, 0, 1))
```

```
print(MA)
```

```
##
```

```
## Call:
```

```
## arima(x = Nile, order = c(0, 0, 1))
```

```
##
```

```
## Coefficients:
```

```
##          ma1  intercept
```

```
##          0.3783    919.2433
```

```
## s.e.    0.0791    20.9685
```

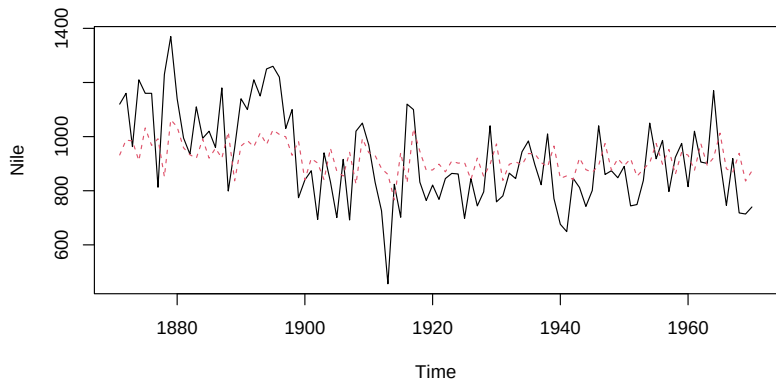
```
##
```

```
## sigma^2 estimated as 23272:  log likelihood = -644.72,  a
```

Estimate the simple moving average model (cont.)

```
# Plot Nile and MA_fit  
ts.plot(Nile)  
MA_fit <- Nile - resid(MA)  
points(MA_fit, type = "l", col = 2, lty = 2)
```

Estimate the simple moving average model (cont.)



Estimate the simple moving average model (cont.)

By fitting an MA model to your Nile data, you're able to capture variation in the data for future prediction.

Based on the plot you've generated, does the MA model appear to be a strong fit for the Nile data?

Answer: No, the MA model does not appear to be a particularly strong fit for the Nile data.

Simple forecasts from an estimated MA model

Now that you've estimated a MA model with your Nile data, the next step is to do some simple forecasting with your model. As with other types of models, you can use the `predict()` function to make simple forecasts from your estimated MA model. Recall that the `$pred` value is the forecast, while the `$se` value is a standard error for that forecast, each of which is based on the fitted MA model.

Once again, to make predictions for several periods beyond the last observation you can use the `n.ahead = h` argument in your call to `predict()`. The forecasts are made recursively from 1 to `h`-steps ahead from the end of the observed time series. However, note that except for the 1-step forecast, all forecasts from the MA model are equal to the estimated mean (`intercept`).

In this exercise, you'll use the MA model derived from your Nile data to make simple forecasts about future River Nile flow levels. Your MA model from the previous exercise is available in your environment.

Simple forecasts from an estimated MA model (cont.)

```
# Make a 1-step forecast based on MA
```

```
predict_MA <-predict( MA)
```

```
# Obtain the 1-step forecast using $pred[1]
```

```
predict_MA$pred[1]
```

```
## [1] 868.8747
```

```
# Make a 1-step through 10-step forecast based on MA
```

```
predict(MA, n.ahead = 10)
```

```
## $pred
```

```
## Time Series:
```

```
## Start = 1971
```

```
## End = 1980
```

```
## Frequency = 1
```

```
## [1] 868.8747 919.2433 919.2433 919.2433 919.2433 919.2433
```

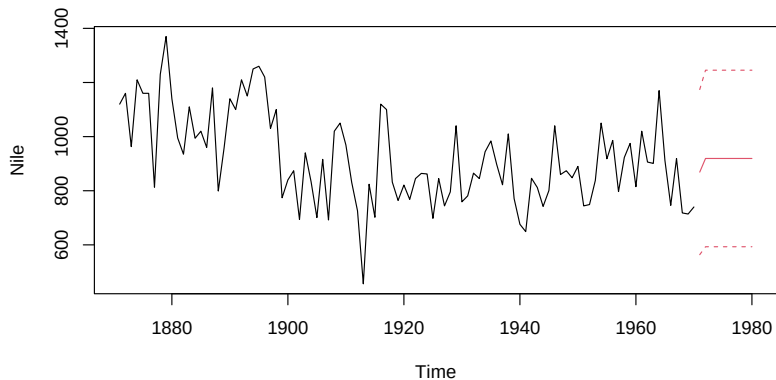
Simple forecasts from an estimated MA model (cont.)

```
## [9] 919.2433 919.2433
##
## $se
## Time Series:
## Start = 1971
## End = 1980
## Frequency = 1
## [1] 152.5508 163.1006 163.1006 163.1006 163.1006 163.1006
## [9] 163.1006 163.1006
```

Simple forecasts from an estimated MA model (cont.)

```
# Plot the Nile series plus the forecast and  
# 95% prediction intervals  
ts.plot(Nile, xlim = c(1871, 1980))  
MA_forecasts <- predict(MA, n.ahead = 10)$pred  
MA_forecast_se <- predict(MA, n.ahead = 10)$se  
points(MA_forecasts, type = "l", col = 2)  
points(MA_forecasts - 2*MA_forecast_se,  
       type = "l",  
       col = 2,  
       lty = 2)  
points(MA_forecasts + 2*MA_forecast_se,  
       type = "l",  
       col = 2,  
       lty = 2)
```

Simple forecasts from an estimated MA model (cont.)



Note that the MA model can only produce a 1-step forecast. For additional forecasting periods, the `predict()` command simply extends the original 1-step forecast. This explains the unexpected horizontal lines after 1971.

Section 18

Model Comparison & Forecasting

AR vs MA models

As you've seen, autoregressive (AR) and simple moving average (MA) are two useful approaches to modeling time series. But how can you determine whether an AR or MA model is more appropriate in practice?

To determine model fit, you can measure the Akaike information criterion (AIC) and Bayesian information criterion (BIC) for each model. While the math underlying the AIC and BIC is beyond the scope of this course, for your purposes the main idea is these indicators penalize models with more estimated parameters, to avoid overfitting, and smaller values are preferred. All factors being equal, a model that produces a lower AIC or BIC than another model is considered a better fit.

Compare AR and MA models (cont.)

To estimate these indicators, you can use the `AIC()` and `BIC()` commands, both of which require a single argument to specify the model in question.

In this exercise, you'll return to the Nile data and the AR and MA models you fitted to this data. These models and their predictions for the 1970s (`AR_fit`) and (`MA_fit`) are depicted in the plot on the right.

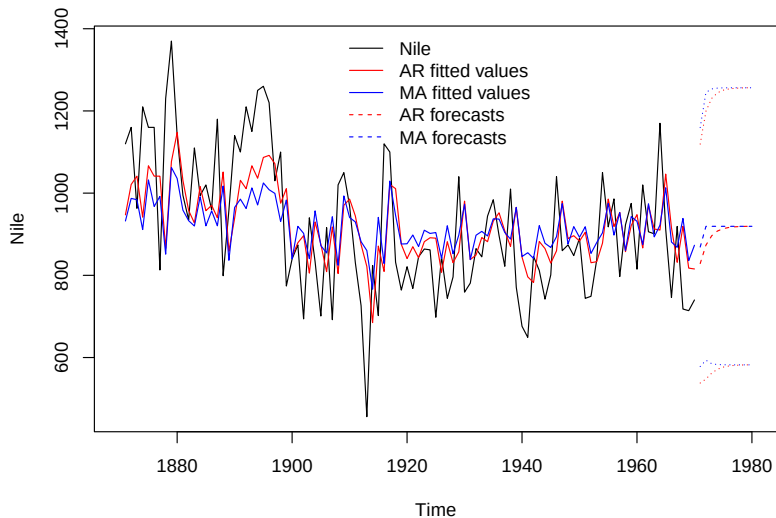
```
AR <- arima(Nile, order = c(1, 0, 0))
```

```
AR_fit <- Nile - residuals(AR)
```

```
MA <- arima(Nile, order = c(0, 0, 1))
```

```
MA_fit <- Nile - residuals(MA)
```


Visual understanding (cont.)



Find correlation between AR_fit and MA_fit

```
cor(AR_fit, MA_fit)
```

```
## [1] 0.9401765
```

```
# Find AIC of AR
```

```
AIC(AR)
```

```
## [1] 1285.904
```

```
# Find AIC of MA
```

```
AIC(MA)
```

```
## [1] 1295.442
```

```
# Find BIC of AR
```

```
BIC(AR)
```

```
## [1] 1293.72
```

Find correlation between AR_fit and MA_fit (cont.)

```
# Find BIC of MA  
BIC(MA)
```

```
## [1] 1303.257
```

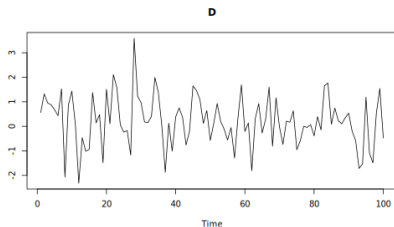
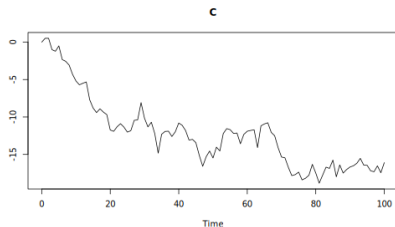
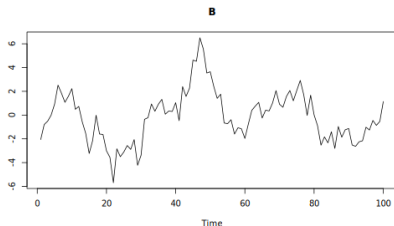
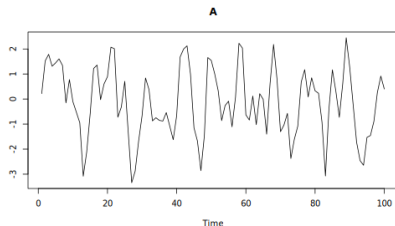
Although the predictions from both models are very similar (indeed, they have a correlation coefficient of 0.94), both the AIC and BIC indicate that the AR model is a slightly better fit for your Nile data.

Name that model by time series plot

Now that you've simulated, fitted, and generated forecasts based on several models, you should have a good idea of what each model looks like and where it is most useful.

A time series from each of the four models we have considered in this course was simulated and they are shown in one of the four panels in the plot on the right. They include the white noise (WN), random walk (RW), autoregressive (AR), and simple moving average (MA) models. Match each time series plot with one of our models WN, RW, AR, MA.

Name that model by time series plot (cont.)



Answer: ✓ (A) MA, (B) AR, (C) RW, (D) WN

Name that model by time series plot (cont.)

Series A shows short-run dependence but reverts quickly to the mean, so it must be the MA model. Series B and C are consistent with AR and RW, respectively. Series D does not show any clear patterns, so it must be the WN model.

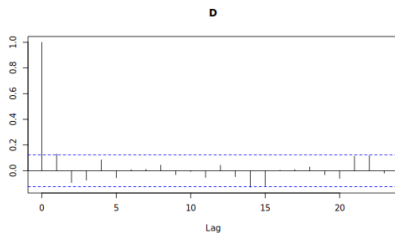
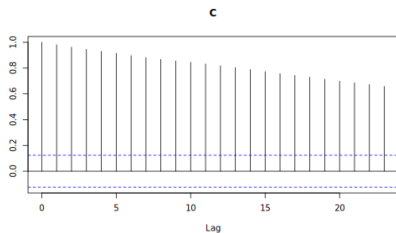
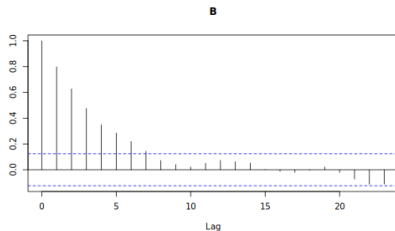
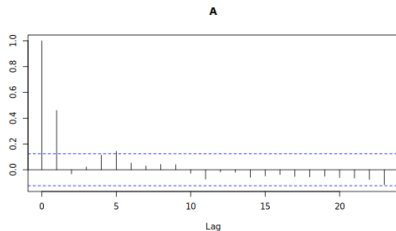
Name that model by ACF plot

Finally, the ACF plot for each model provides additional useful information about how each model operates (and, therefore, when you should use each model). It is also helpful to be able to identify the model based on the ACF plot.

A time series from each of the four models you have considered in this course was simulated and their sample autocorrelation functions (ACF) are shown in one of the four panels in the adjoining figure. They include the white noise (WN), random walk (RW), autoregressive (AR), and simple moving average (MA) models.

Match each sample ACF plot with one of our models WN, RW, AR, MA.

Name that model by ACF plot (cont.)



Answer: ✓ (A) MA, (B) AR, (C) RW, (D) WN

Section 19

Final revision

Data structure

`start()` · `end()` · `frequency()` · `cycle()` · `ts()`

Transformation

Log → stabilize changing variance

Difference → remove level trend

Seasonal difference → remove repeating seasonal structure

Dependence

ACF → identify serial dependence and guide model selection

The lecture in one page (cont.)

Models

WN → no serial dependence

RW → unit-root / cumulative noise

AR → dependence on past values

MA → dependence on past shocks

Model choice

AIC/BIC: smaller is preferred, all else equal.

A practical diagnostic checklist

1. Plot the series

Look for trend, seasonality, changing variance, and unusual observations.

2. Check the time structure

Confirm start, end, frequency, and missing observations.

3. Transform if needed

Use log and/or differencing when the structure calls for it.

4. Inspect the ACF

Look for persistence, oscillation, seasonality, or near-zero dependence.

5. Fit candidate models

Use `arima()` and keep the model parsimonious.

6. Compare and validate

Use fitted values, residual behavior, AIC/BIC, and forecast performance.

Golden rule: *A good time-series model should explain the dependence without leaving systematic structure in the residuals.*

- Q1.** Which model is non-stationary by construction?
- Q2.** What does a slowly decaying ACF suggest?
- Q3.** Which transformation removes a level trend?
- Q4.** For an MA(1), after which lag should the theoretical ACF cut off?
- Q5.** When comparing otherwise similar models, do we prefer a higher or lower AIC/BIC?

Time series analysis is a sequence of decisions

Understand the time → reveal the pattern → make the series suitable → model dependence → check the model → forecast

The goal is not simply to fit a model.

The goal is to understand the temporal structure well enough to make defensible conclusions.

Thank You